

AIM

Complete Research Architecture & Project Handbook

Research, conclusions, decisions, implementation plan, and VIT
machine-learning-lab compute audit

Evidence cut-off: 24 September 2026
Prepared as a buildable, falsifiable research proposal
Hardware inventory is user-reported and must be verified

Hardware update and interpretation

The user reports approximately **70–84 machines** in the VIT machine-learning lab, each described as having a 13th-generation Intel Core i9, 32 GB RAM and Intel UHD Graphics 770. If the reported node count and per-node configuration are consistent, total installed memory is approximately **2.24–2.688 TB** across separate hosts. This is distributed RAM, not a unified pool.

This changes the project from a single-workstation plan to a potential CPU-fleet plan. Cluster access, simultaneous availability, exact processor SKUs, node homogeneity, network topology/bandwidth, scheduler, storage, software environment and measured model-training throughput remain unknown. A collection of lab computers is not automatically a schedulable cluster. The report therefore separates user-reported facts from conditional arithmetic and benchmark scenarios.

Recommended immediate use: parallelize corpus ingestion/deduplication, search indexing, symbolic and statistical verification, code execution, simulations, benchmark evaluation, synthetic task generation, and independent inference/rollout jobs. For model training, begin with single-node 100M–300M CPU proxies, measure distributed 1B scaling, and only then decide whether a sharded 7B pilot has a useful wall-clock schedule. Sharding can distribute state; it cannot remove the aggregate compute and inter-node communication requirements. Accelerators remain the preferred route for substantial pretraining.

Illustrative 7B / 140B-token compute: $F \approx 6ND = 5.88 \times 10^{21}$ FLOPs. At hypothetical per-node sustained useful throughput of 10 / 100 / 1,000 GFLOP/s across 70–84 nodes, idealized times are about 222–266 / 22.2–26.6 / 2.22–2.66 years. These are sensitivity calculations, not measured VIT capability; real schedules are longer after communication, availability, validation, checkpointing and utilization.

Contents and evidence-status guide

This handbook contains the executive architecture, detailed research conclusions, training and reward design, full lab audit, scaling calculations, implementation/evaluation plans, decision record, literature and open questions. It closes with the source Charter and Astra Directive supplied by the project owner. Those appendices are source material; they are not independent evidence and were not treated as higher-priority instructions for this research task.

Evidence labels: Established fact = directly supported by a cited primary source or measured input; Evidence-backed recommendation = supported by relevant evidence plus engineering judgment; Hypothesis = falsifiable but unproven; Speculative idea = insufficient evidence to rely on; Unresolved = requires measurement, policy input or research.

Master research architecture

Project research document · source path: documents/AIM_Master_Research_Architecture.md

AIM Master Research Architecture

AIM Research Intelligence Model Research cut-off: 24 September 2026 **Version:** 0.1 — evidence-based proposal, with unresolved hardware-dependent choices **Status:** ready for hardware/data audit and falsification experiments; not a final parameter-scale authorization

Executive decision

Build AIM as a research system rather than a monolithic “scientist” model: an autoregressive researcher, an explicit evidence/research state, a deterministic controller, a small calibrated decision component, independently versioned verifiers, and provenance-aware memory. The distinctive capability to test is whether this system makes *more independently verified, source-supported research progress per unit cost* than a strong retrieval/tool baseline.

Begin with licensed-checkpoint system baselines plus trainable 100M–1B proxies. Use a dense decoder Transformer as the first trainable control. The user reports approximately 70–84 VIT machine-learning-lab systems, each described as 13th-generation Intel Core i9, 32 GB RAM and Intel UHD Graphics 770. If consistent and concurrently available, this implies 2.24–2.688 TB aggregate RAM distributed across hosts, not one addressable memory pool. The fleet may be joinable as a CPU cluster; that capability, network, scheduler, access windows, storage and sustained throughput are unverified. Start with 100M–300M CPU proxies and parallelize data, indexing, verifiers, evaluation, simulation and rollouts. Benchmark 1B distributed scaling before deciding whether a sharded 7B run has a useful schedule. Aggregate memory alone neither proves nor disproves practical 7B training; measured end-to-end throughput, data, budget and cluster reliability determine feasibility. Accelerator access remains preferred for substantial pretraining. Do not choose 14B/32B/70B or a 100B–400B MoE on aspiration alone.

Use SFT → DPO preference baseline → isolated outcome-based RLVR → separately trained/calibrated decision policy → integrated loop only after factorial ablations. Do **not** begin with $w_h R_h + w_v R_v + w_c R_c + w_p R_p$. Correctness, subjective utility, calibrated forecasts and process traces have different semantics and error models. Use hard constraints/release gates for critical correctness/provenance, separate proper-score calibration, and Pareto reporting. Treat the September 2026 RLCD work as an interesting interface/training claim whose public reproducibility is currently insufficient to adopt as an algorithm.

How to read this proposal

The attached Charter supplies the scientific vision and candidate architecture. The Astra Directive supplies the research protocol and requested artifacts. Both are hypotheses/requirements for investigation; neither is evidence that its claims or scale plan are correct. This report labels established evidence, recommendation, hypothesis, speculation and unknown separately. “Verified” always means a named check passed within a stated scope, not absolute truth.

1. Mission and measurable system claim

Target workflow: define a question → identify assumptions and unknowns → retrieve and read evidence → generate competing hypotheses → derive distinct predictions → run computation/experiments/proofs → compare results and contradictions → revise beliefs → write an artifact with claim-level provenance.

Primary outcome proposed for AIM: independently verified supported research progress per unit compute/tool cost. Report its components rather than conceal them in a composite: resolved subtasks, verified claim fraction, citation support, successful/reproducible experiments, correctly rejected hypotheses, contradictions found, calibration of resolved forecasts, unsupported-claim rate, human usefulness, and cost/time. Validate the metric with researchers before optimizing it.

This does not promise discovery of new scientific truths. Scientific novelty, causal validity and future empirical truth often lack immediate machine-checkable labels. Keep these human-reviewed or longitudinal tasks separate from exact verification.

2. Architecture choices

Proposed system: researcher + controller + judge + verifier ensemble

Researcher: decoder language model proposes structured actions, claims, hypotheses, predictions, analyses and final prose.

Controller: deterministic shell enforces schemas, permissions, limits, provenance and state transitions. A learned router may rank legal candidate actions.

Decision/judge model: small separate model initially; emits a typed distribution for explicit tasks such as source-support status, contradiction presence, continue/retrieve/verify/abstain, or hypothesis selection. It cannot declare unobserved claims true.

Verifier ensemble: code/tests; symbolic math; theorem prover; simulation; statistical recomputation; source/version/span and citation checking; human experts where needed.

Research memory: relational provenance ledger, evidence graph, object storage, lexical + vector retrieval. Vector similarity is discovery, not proof.

Human review: for high impact, unresolved conflicts, novelty, invalid/missing source authority or out-of-distribution decisions.

Alternatives

Candidate	Pros	Risks	Recommendation
A. Shared backbone + decision head	Efficient, shared knowledge, lower latency	Calibration/objective conflict; head inherits shared errors	Test against separate model; not assumed sufficient
B. Researcher + calibrated judge	Independent objective/data/update; modular	Correlated blind spots, latency, judge bottleneck	Best initial integration
C. Large MoE + separate judge + verifiers/router	Sparse capacity and role separation	Total memory/optimizer burden, all-to-all, complex failure surfaces	Future only if hardware and proxy studies justify it
Retrieval/tool system with existing model	Fastest useful baseline; tests system value	Checkpoint/license dependency; no new base model	Mandatory baseline, not final scientific contribution by itself

3. Foundation model architecture and size

Scale decision: explicitly unresolved; preliminary 70–84-node lab inventory supplied

The Directive’s dense 7B/14B/32B/70B and MoE 100B–400B+ ranges are candidate design space, not verified feasible targets. VIT’s reported 70–84 CPU systems change the memory and parallel-workload picture, but provide no measured training throughput. Compute estimates below are sensitivity cases; only cluster inventory and representative multi-node benchmarks can establish feasibility. A public frontier report proves specialized infrastructure, not VIT capability.

Conditional dense baseline

If measured accelerator access permits, profile a conventional decoder-only pre-norm Transformer near 7B: provisional 32 layers, hidden width 4096, 32 query heads, 8 KV heads, dimension 128, SwiGLU, RMSNorm, RoPE, GQA. Exact parameter count must be recalculated from config/vocabulary/embedding tie before run; this shape is near, not guaranteed to equal, 7B. Start at 4K–8K context; 16K/32K staged extension requires explicit retrieval comparison and activation/KV budget.

Tokenizer comparison: byte-fallback BPE or unigram, candidate vocabulary 32K–64K; evaluate domain compression and lossless treatment of Unicode, LaTeX, scientific notation, code, tables and multilingual content. Freeze only after measurement.

Baseline objective: causal next-token NLL. AdamW is the reference optimizer; benchmark Muon under same data, compute and tuning budget before selection. BF16 only where supported and numerically validated; FP32 for CPU proxy; FP8 only on capable accelerator after validation. Use activation checkpointing, fused attention where available, and sharded optimizer/model states as needed.

Dense vs MoE

Dense activates all weights per token, simplifies load balancing and is robust to weak network. MoE increases total capacity while active compute tracks selected experts approximately; **weights, optimizer states, checkpoint size track total parameters**, while expert token dispatch adds all-to-all traffic. Load imbalance, capacity overflow, router collapse, stragglers and inference residency can erase FLOP savings. DeepSeekMoE’s fine-grained/shared experts are an experimentally supported option, not a universal rule. First test modest 8–16-expert MoE against dense at matched active compute, token count and quality. Track load, route entropy, expert utilization, wall time, communication, cross-domain transfer, inference latency. Do not pin experts to domains initially; specialization/fragmentation is a hypothesis.

Alternatives in attention/sequence modeling

- **GQA:** strong KV-cache compromise; test quality and cache at same compute.
- **MLA:** DeepSeek reports reduced KV cache; more implementation complexity and kernel dependence; compare end-to-end.
- **RoPE:** portable baseline; YaRN/other extension only after long-context comparisons.
- **FlashAttention-2:** IO-aware exact attention, useful where hardware kernels support it; not a model architecture replacement.
- **State-space/hybrid:** credible alternative for sequence efficiency; defer unless measured attention/context bottleneck and compare on evidence recall/synthesis.
- **Multi-token prediction:** reported by DeepSeek-V3; treat as auxiliary objective ablation, not assumed quality gain.

4. Scaling and compute

The standard first-order dense estimate is $F \approx 6ND$, where N is parameters and D training tokens. This omits implementation overhead, attention nuance, evaluations, failures, post-training and networking. MoE approximation uses active parameters for token FLOPs but total parameters for memory/optimizer/storage and routing communication. Chinchilla’s compute-optimal findings are a reference, not a universal 20-token law; modern data quality, repeated data and inference demand can move the optimum.

Candidate	Illustrative token budget	FLOPs $6ND$	Time at 1 PFLOP/s sustained
7B dense	140B	5.88e21	68 days
14B dense	280B	2.35e22	272 days
32B dense	640B	1.23e23	3.90 years
70B dense	1.4T	5.88e23	18.7 years
20B-active MoE	400B	4.80e22	1.52 years
32B-active MoE	1T	1.92e23	6.09 years

At 100 TFLOP/s sustained, multiply times by 10; at 10 TFLOP/s, by 100. For the reported 70–84 nodes, illustrative useful per-node throughput scenarios of 10/100/1,000 GFLOP/s correspond to 0.70–0.84/7.0–8.4/70–84 TFLOP/s aggregate and roughly 222–266/22.2–26.6/2.22–2.66 years for the 7B run, assuming perfect linear scaling. These are not measured estimates and omit communication, downtime, utilization and other overhead; actual schedule would be longer. Sensitivity is linear in token count and inverse in sustained throughput. Aggregate RAM is not pooled.

BF16 weights require $2N$ bytes: 7B≈14GB, 14B≈28GB, 32B≈64GB, 70B≈140GB, MoE 300B≈600GB. A conservative 16B/parameter Adam training-state estimate gives 7B≈112GB, 14B≈224GB, 32B≈512GB, 70B≈1.12TB, 300B≈4.8TB before activations/workspaces. Sharding distributes these bytes but does not remove them. KV cache for standard K/V $\approx 2 \times \text{layers} \times \text{sequence} \times \text{KV_heads} \times \text{head_dim} \times \text{bytes}$; a 32-layer, 8-KV-head, 128-dimension BF16 model at 32K is roughly 4 GiB per sequence before overhead.

Scale recommendation: 100M–300M single-node CPU proxies first; use the potential fleet for parallel data/verifier/evaluation work immediately, then measure a distributed 1B pilot. Gate sharded 7B experiments on measured scaling, schedule, data and budget; dedicated accelerators are preferred for substantial pretraining. 14B/32B only after scaling curves, licensed tokens, run allocation, storage and inference plan justify them; 70B/MoE stretch only. Full calculation and assumptions: [scaling report](../research/scaling/scaling-and-compute-analysis.md) and [updated VIT hardware audit](../research/hardware/vit-ml-lab-audit.md).

5. Data program

Build licensed, versioned corpus layers:

- **A Knowledge:** permitted reference/educational material, open scholarly full texts only when their specific rights permit, technical documentation.
- **B Structured reasoning:** math, proofs, code/tests, derivations, experiments and protocols.
- **C Research process:** question/search/evidence/hypothesis/test/result/conclusion traces.
- **D Negative/corrective:** wrong proof/citation/statistic/experiment paired with diagnosis and repair.
- **E Synthetic environments:** math, causal and scientific worlds with exact hidden ground truth and randomized generators.

Access does not establish model-training rights. OpenAlex metadata are CC0 but linked work/full-text rights can differ; Semantic Scholar API has its own agreement. Track rights at source/object level, including attribution/share-alike/noncommercial/opt-out; default unknown rights to exclude or retrieval-only pending legal review. Record manifest, hash, transformations, parser, license, domain/date, dedup cluster, contamination lineage and resulting runs. Hold out entire source families, dates and synthetic generator families.

Do not set final mixture percentages before inventory and pilot. Compare broad high-quality corpus and reasoning-heavy continued-pretraining mixtures at equal compute while preserving general-language anchor. Synthetic teachers are not labels: accept only outputs checked by independent tools or expert adjudication.

6. Training: RLHF, RLVR, RLCD, process feedback

Signals and math

- **LM:** $-E \sum_t \log \pi_\theta(x_t|x_{<t})$; models text distribution, not truth.

- **Preferences/RLHF:** pairwise Bradley–Terry $P(y_w > y_l) = \sigma(r\phi(x, y_w) - r\phi(x, y_l))$; PPO optimizes clipped policy objective with KL to reference. DPO is a simpler offline preference baseline under its derivation assumptions.
- **RLVR:** objective outcome reward from trusted math/code/symbolic/proof/simulation/statistics checks; compare REINFORCE/PPO and GRPO. GRPO uses within-prompt relative rewards and has normalization/length/difficulty effects; DAPO/Dr. GRPO variants require controlled reproduction.
- **RLCD-style decisions:** typed action/outcome distribution and calibrated scores for defined, resolved tasks. Optimize Brier or log score; evaluate on held-out domains/shifts. Abstention is an action chosen from expected task costs.
- **Process reward:** step label or value estimate may guide search; it is not proof. PRM evidence is strongest in math and a particular best-of-N setup. Keep it auxiliary/bounded and evaluate plausible wrong traces.

Brier $(p-y)^2$ and log loss are proper scoring rules under standard assumptions; ECE is a bin-dependent diagnostic, not a proper loss. Calibration is for a specified event/reference class/horizon. “Claim is entailed by these spans” is different from “hypothesis will replicate next year.” Unresolved scientific truth is censored/unresolved, not a fabricated binary label.

Recommended training order and controls

Base pretrain and scaling proxies; 2. mid-train science/math/code and context only if measured; 3. Research SFT for structured actions/evidence and communication; 4. DPO preference baseline; 5. isolated verifier-grounded RLVR; 6. separately trained decision policy with supervised proper score then RL only where action returns are observable; 7. optional process reward; 8. integrated state/action RL study.

This order is a pragmatic experiment, not a claim sequential is optimal. Compare interleaved/joint methods after components work. Avoid a single scalar sum: make source/provenance and critical verification release constraints; separately optimize preference and decision calibration; report Pareto front. If reward scalarization is tested, normalize using frozen training statistics, specify weights, perform sensitivity analysis and monitor independent outcomes. Stop on proxy reward/outcome divergence, reward hacking or held-out regression.

7. Research agent, memory, verifiers

The `ResearchState` contains question/scope, assumptions, facts/unknowns, sources, evidence spans, hypotheses/predictions, experiments, results, contradictions, forecasts/confidence, open tasks, provenance and final claims. Represent state as append-only events with supersedes/retracts edges and a materialized relational/graph view. Store originals/results as immutable objects; lexical/vector indices retrieve candidates. Each claim links source version → exact evidence span → derivation/transformation → verifier result → confidence target.

Actions: SEARCH, READ, EXTRACT, COMPARE, CALCULATE, CODE, RUN, SIMULATE, PROVE, CHALLENGE, HYPOTHESIZE, VERIFY, ABSTAIN, REVISE, WRITE. Each has schema, allowed resources, timeout, cost, result type and error state. Deterministic shell enforces contracts; learned researcher/router proposes actions. For candidate action use expected information value, impact and cost; stop at marginal value below cost, hard budget, or human-review boundary.

Verifier stack: schema and source identity; code compile/tests/property tests in fresh sandbox; SymPy/numeric checks with assumptions/tolerances; Lean/Isabelle proof where formalization makes sense; simulation/statistical recomputation; citation entailment and contradiction checks; independent small judge; human review. Preserve raw output/version/hash. “VERIFIED” means that specific test passed on that artifact and scope. A citation is not evidence unless the cited span supports the claim.

8. Failure modes and defenses

Failure	Defense / measurement
Reward-model overoptimization, verbosity/style gaming	DPO/PPO comparison; hidden human audit; length-controlled pairs; independent outcomes; KL and early stop
Verifier loopholes / test leakage	Fresh sandbox; hidden/rotating tests; independent implementations; metamorphic testing; red-team; grader audits
Confidently wrong judge	Proper-score held-out evaluation, domain/time shift, independent verifier ensemble, abstain/review, human audit
Fabricated/weak citations	Resolve source/version, exact span and entailment; unsupported-claim metric; source quality and contradiction review
Calibration collapse under distribution shift	Per-domain/time/source curves, Brier/log, selective risk–coverage, OOD review gates
Process reward values plausible invalid steps	Outcome-conditioned labels, terminal verifier, adversarial near-miss cases, bounded use
MoE router imbalance / fragmentation	All-to-all/load metrics, cross-domain transfer, matched dense baseline; defer if not measured viable
Context loses evidence / retrieval bias	Exact provenance, hybrid search, retrieval ablation, evidence recall, claim graph, contradiction injection

Failure	Defense / measurement
Data contamination/rights failures	Rights manifest, dedup, sealed sets, source lineage, exclusion defaults and removal process
Scientific delay/no ground truth	Forecast ledger with target/horizon/resolution; unresolved/censored state; expert longitudinal review
Demo/research theater	Preregistered evaluation, negative runs, independent reproduction, open configs/artifacts where permitted

9. Evaluation framework

Create sealed, source-family/generator/time-held-out suite before training. Measure knowledge and temporal facts; math exact answers, derivations and proof validity; code hidden tests/complexity/security; scientific hypotheses and experimental design; literature synthesis/citation support/contradiction discovery; calibration and selective action; multi-step task completion, state retention, provenance and cost; scholarly communication by blind experts. Cover math, CS, physics, quantum information, astronomy, chemistry, biology, psychology/cognitive science, statistics, engineering, economics, philosophy/logic, research methodology and writing. Report each domain separately.

Public diagnostic anchors: GPQA, MATH/FrontierMath when licensed/available, Lean tasks, SWE-bench variants, SciCode, ALCE, PaperBench and long-context retrieval benchmarks. Public scores are vulnerable to contamination and are not an overall research-intelligence metric. LLM graders require human validation, blinding, verbosity/position tests and inter-rater analysis. Primary measures: claim support precision/recall, verified result correctness, unsupported-claim rate, Brier/log score, risk-coverage and verified research progress/cost. Use confidence intervals, multiple seeds where possible, budget-matched baselines and report all failures.

10. Hardware and implementation plan

Audit first

Inventory CPU SKU/instructions/cores/NUMA/RAM/storage, GPU/VRAM/runtime, job limits, scheduler, uptime, network topology/RDMA/bandwidth/latency, quotas. Benchmark representative forward/backward/update, tokenizer/decompression, inference at 1K–32K, all-reduce and MoE all-to-all, checkpoint/resume, full-day availability. Record p50/p95, utilization and repeats. The user reports ~70–84 VIT lab systems, each described as 13th-generation Intel Core i9, 32 GB RAM and UHD Graphics 770. This preliminary inventory implies 2.24–2.688 TB aggregate installed RAM if consistent, distributed across nodes. It does not establish SKU consistency, simultaneous access, schedulable cluster status, network/storage performance, throughput or additional GPUs. Inventory and benchmark each node class; measure representative single-node and multi-node training, inference, all-reduce, checkpointing and availability. The Codex host separately reported Apple M3 integrated GPU (10 cores); it is not part of the VIT estimate.

If GPU nodes are present: dense DP/FSDP first; add TP/PP/context parallel only as memory/topology require; EP only after MoE test. CPU nodes provide valuable corpus preparation, dedup, search, verifier execution, theorem/symbolic work, simulation, reward generation, evaluation and rollout orchestration even if they cannot economically pretrain final scale. Actual contribution depends on benchmark and job constraints.

11. Milestones and research value

0. Lab access + hardware benchmark + rights audit. **1.** Data manifest, tokenizer studies, evidence graph, verifiers, baseline retrieval-agent. **2.** 100M/300M/1B dense scaling/objective proxies. **3.** SFT/preferences/RLVR/RLCD factorial post-training. **4.** Long-horizon loop with contradiction, tool failure and held-out source tests. **5.** Choose 7B/14B/32B only from measured scaling/budget; fund accelerator access if needed. **6.** External replication; publish results and limitations.

An AIM contribution is scientifically meaningful if it shows reproducible causal gains over a strong retrieval/tool and same-model baseline at fixed budget, with component ablations, independent graders, held-out domains/generators, and no unacceptable calibration/factuality regression. “We used RLHF + RLVR + RLCD” or “we trained a large MoE” is not sufficient.

12. References and claim anchors

Ouyang et al., [Training language models to follow instructions with human feedback](#), 2022.

Rafailov et al., [Direct Preference Optimization](#), 2023.

Shao et al., [DeepSeekMath](#), 2024.

Guo et al., [DeepSeek-R1](#), 2025.

Yu et al., [DAPO](#), 2025; [Dr. GRPO analysis](#), 2025.

Lightman et al., [Let's Verify Step by Step](#), 2023; [PRM800K data/code](#).

Gao et al., [ALCE](#), EMNLP 2023.

Dai et al., [DeepSeekMoE](#), 2024; [DeepSeek-AI](#), [DeepSeek-V3](#), 2024.

Yang et al., [Qwen3](#), 2025; [GLM-4.5](#), technical report, 2025; Team OLMo, [OLMo 3](#), 2025.

Hoffmann et al., [Training Compute-Optimal Large Language Models](#), 2022; Kaplan et al., [Scaling Laws](#), 2020; Sardana et al., [Beyond Chinchilla-Optimal](#), 2024.

Liu et al., [Muon is Scalable for LLM Training](#), 2025.

Ainslie et al., [GQA](#), 2023; Dao, [FlashAttention-2](#), 2023/ICLR 2024; Peng et al., [YaRN](#), 2023.

Guo et al., [On Calibration of Modern Neural Networks](#), 2017; Kadavath et al., [Language Models \(Mostly\) Know What They Know](#), 2022; [LLMs Must Be Taught to Know What They Don't Know](#), NeurIPS 2024.

TypeSafe AI, [Introducing System One Models & Jev](#), 15 Sep 2026; [RLCD description](#). Company announcement; reproducible method not disclosed in reviewed sources.

Yao et al., [ReAct](#), 2022; Schick et al., [Toolformer](#), 2023.

Rein et al., [GPQA](#), 2023; Starace et al., [PaperBench](#), 2025.

[Megatron Core parallelism documentation](#); [OpenAlex API and license docs](#), [Semantic Scholar data license](#).

See the supporting literature matrix, architecture/algorithm reviews, hardware report, data strategy and decision record for expanded comparisons, source quality and unresolved questions.

Research program overview

Project research document · source path: research/README.md

AIM research program

Evidence cutoff: 24 September 2026. **Status:** research architecture proposal; not a claim that the VIT cluster has been audited or that a final model size is authorized.

This repository separates the user's request (produce a reproducible, sourced research package) from the attached Charter (scientific vision and hypotheses) and Astra Directive (operational protocol and requested deliverables). Those attachments are project requirements, not evidence that their architectural propositions are correct.

Decision summary

Build and compare a dense decoder Transformer first. The user reports approximately 70–84 VIT lab systems, each described as 13th-gen Intel Core i9, 32 GB RAM and UHD Graphics 770. If consistent and concurrently available, that is 2.24–2.688 TB aggregate RAM across separate hosts and a potential CPU cluster, not shared memory. Node count/spec consistency, exact SKUs, network, scheduler, storage, policies and sustained throughput are unverified. Use 100M–300M single-node CPU proxies first; measure a distributed 1B pilot and cluster workloads. A sharded 7B run is neither ruled out by aggregate memory alone nor justified by it: select only after a measured end-to-end pilot and schedule/budget analysis. Use accelerators if accessible for serious pretraining. The supplied inventory is a report, not an independent audit. The local Codex host's Apple M3 observation is unrelated to VIT.

Build the initial research system around an existing openly available checkpoint as a systems baseline where its license permits, plus an independently trainable 100M–1B model for controlled learning experiments. The contribution should be a verified research loop and a falsifiable training study, not parameter count.

Use one autoregressive researcher with structured tool contracts, deterministic provenance storage, independent domain verifiers, and initially a small separate correctness/decision model. Treat a shared decision head as an ablation. RLHF, RLVR, and calibrated decisions serve different tasks and should not be naively summed: start with SFT, preference learning, isolated verifier-based RL, and a separately scored decision policy; integrate only after factorial ablations.

Document map

- `literature/literature-matrix.md` — source index and evidence status.
- `architecture/state-of-the-art-architecture-review.md` — alternatives and proposed model family.
- `rl/rlhf-rlvr-rlcd-review.md` — objective mathematics, RLCD evidence limits, sequencing.
- `rl/reward-and-verifier-design.md` — verifier ensemble and anti-gaming controls.
- `data/data-strategy.md` — corpus tiers, licenses, provenance and synthetic data.
- `scaling/scaling-and-compute-analysis.md` — transparent FLOP and memory scenarios.
- `hardware/vit-ml-lab-audit.md` — what is known, what is not, and an executable audit protocol.
- `agentic-research/research-loop-design.md` — controller, state and evidence graph.
- `evaluation/evaluation-and-benchmark-plan.md` — metrics, splits and gates.
- `open-questions/open-research-questions.md` — explicit hypotheses and deciding experiments.
- `documents/final-design/` — architecture, training, evaluation and implementation specifications plus decision record.
- `documents/AIM_Master_Research_Architecture.md` — consolidated master artifact.

Epistemic labels

- **Established evidence:** directly reported in a primary paper, official technical report, implementation, or documentation; does not mean universal truth.
- **Recommendation:** reasoned design choice supported by evidence and project constraints.
- **Hypothesis:** testable claim not established for AIM.
- **Speculation:** plausible idea with weak direct evidence.
- **Unknown:** necessary measurement or source disclosure is absent.

Reproducibility rule

Record exact source URL, version/date, license, retrieval time, transformations, hashes, benchmark split, software versions, random seeds, hardware, tokens, optimizer, run duration, and checkpoints. Freeze a held-out evaluation set before training. Archive failed runs and negative results. Revisit any current model or service specification against its primary source before implementation.

VIT machine-learning-lab audit

Project research document · source path: research/hardware/vit-ml-lab-audit.md

VIT ML lab hardware audit

Status: project-reported fleet inventory, not an independent physical audit or performance benchmark. **Date recorded:** 24 September 2026.

Inventory reported by AIM

The lab has approximately **70-84 machines** described as having the following per-machine specifications:

Component	Per machine	Fleet-level interpretation	Audit state
CPU	13th-generation Intel Core i9	70-84 CPUs; exact aggregate core/thread count cannot be established until exact SKUs are known	Generation/family stated; exact SKU, desktop/mobile variant, core topology, instruction support, clocks/power not confirmed
System memory	32 GB RAM	2.24-2.688 TB raw installed RAM across 70-84 nodes	Capacity stated; whether each has 32 GB usable, speed/channels and reservation remain unverified
Graphics	Intel UHD Graphics 770	Integrated graphics in each reported system; shares host memory, not 70-84 discrete GPUs with independent VRAM	Model stated; drivers, framework compute support and useful ML throughput untested
Count	70-84 machines	Potential CPU cluster if simultaneous access and cluster policies permit	Range supplied by user; exact count and availability schedule unknown
Storage	Not stated	Aggregate storage cannot be calculated	Device, free capacity, local/shared filesystem, quotas and throughput unknown
Network	Not stated	Cluster scaling cannot be predicted	Topology, NIC/switch rates, oversubscription, latency, RDMA and routing unknown
Scheduler / cluster software	Not stated	A connected fleet is not necessarily a schedulable distributed cluster	OS, admin permission, scheduler, MPI/runtime and job limits unknown

The 2.24-2.688 TB figure is a simple aggregate ($\text{node count} \times 32 \text{ GB}$) and **not** a unified memory pool. RAM is physically distributed. A distributed sharded training system can use aggregate memory only if the software, process layout, access rights and network support it; model states, activations, caches and communication buffers still have to fit per node. Data-parallel copies can replicate rather than pool model state.

The exact CPU SKU matters: 13th-generation Core i9 includes desktop and mobile variants with different core counts, power envelopes and memory behavior. Intel's [13th Gen desktop product brief](#) describes multiple variants; it does not identify the VIT SKU. Intel identifies UHD 770 as integrated Xe-based graphics, not a discrete datacenter accelerator with dedicated high-bandwidth VRAM ([Intel brief](#), [graphics API support](#)). Do not count each integrated GPU as a CUDA GPU or assume that it provides an efficient transformer training backend.

CPU distributed processes are technically available in standard software: PyTorch documents Gloo for CPU collectives, while Intel provides MPI/oneCCL libraries for compatible cluster communications ([PyTorch distributed](#), [Intel MPI](#), [Intel oneCCL guide](#)). This establishes software options, **not** that the lab hosts have those packages installed, are interconnected, or can scale efficiently.

Updated feasibility conclusion

This is potentially a substantial CPU fleet, not one workstation. It can make a meaningful AIM contribution through highly parallel data preparation, retrieval/indexing, simulation, symbolic and statistical verification, code tests, synthetic environment generation, benchmark execution, and independently batched CPU inference/rollout jobs. Those workloads often parallelize across independent tasks with less cross-node communication than model training.

For full model training the conclusion is more cautious. A 7B model's BF16 weights are about 14 GB per copy; an Adam-style training-state planning estimate is about 112 GB before activations/workspaces. This exceeds a node's 32 GB. Aggregate raw fleet RAM could hold sharded model states on paper, but CPU FSDP/ZeRO must repeatedly communicate shards; a high-latency or oversubscribed LAN can dominate step time. The fleet might run small distributed CPU training if interconnect and permission allow, but **the user's node count and RAM alone do not establish a practical 7B training budget**.

Illustrative 7B compute scenarios

At 7B parameters and 140B tokens, first-order dense pretraining compute is about 5.88×10^{21} FLOPs (6ND). To show sensitivity, the table assumes perfect linear scaling, 100% availability and an effective *end-to-end transformer training throughput per node* of 10 GFLOP/s, 100 GFLOP/s or 1 TFLOP/s. These are scenario inputs, **not benchmark results or claims about i9 performance**. Real wall time is longer after communication, thermal limits, scheduler downtime, checkpoints, validation and non-scaling work.

Nodes	Assumed sustained useful throughput per node	Aggregate ideal throughput	Idealized 7B/140B-token duration
70	10 GFLOP/s	0.70 TFLOP/s	~266 years
84	10 GFLOP/s	0.84 TFLOP/s	~222 years
70	100 GFLOP/s	7.0 TFLOP/s	~26.6 years
84	100 GFLOP/s	8.4 TFLOP/s	~22.2 years
70	1 TFLOP/s	70 TFLOP/s	~2.66 years
84	1 TFLOP/s	84 TFLOP/s	~2.22 years

Even the 1 TFLOP/s-per-node row is only a hypothetical sensitivity point until a representative full training benchmark demonstrates it. The system is likely valuable for AIM, but the first scientific scale target should be selected from measured tokens/second, not this sensitivity table.

Likely cluster topology and work allocation

If administrators permit, use an initial CPU cluster as a **parallel research/evaluation platform**. Keep the evidence database, object storage and scheduler centralized or appropriately replicated; run independent worker jobs across nodes. First compare: (A) embarrassingly parallel jobs such as parsers/verifiers; (B) multi-process CPU inference; (C) all-reduce for a tiny training proxy; and (D) FSDP/ZeRO-sharded training only if C is healthy. Start with PyTorch Gloo or supported MPI/oneCCL only after confirming the installed operating system/runtime and fabric. Do not select MoE: all-to-all expert routing is especially sensitive to bandwidth and latency, while there are no dedicated GPU accelerators stated.

Recommended usage order:

Data ingestion, rights manifests, deduplication, paper parsing, tokenizer experiments and retrieval-index construction.

CPU verifiers: SymPy, compilers/tests, formal methods, statistics and simulations, with sandbox and resource limits.

Synthetic research worlds, hidden-test generation, evaluation, adversarial red-teaming and human-label triage.

Batch inference/rollouts and small 100M-300M training proxies, measuring throughput and useful concurrency.

Distributed 1B experiments only if a 24-hour measured run is stable, within memory and useful budget.

Treat a 7B CPU cluster run as a research systems experiment only after measured cost/speed indicates meaningful progress; seek accelerator allocation for serious 7B+ base pretraining.

Required audit and benchmark

Inventory every node class: count actually schedulable together, exact CPU SKU, P/E cores, threads, SIMD/AMX flags, NUMA, sustained clock, memory speed/channels, storage and OS.

Record admin permissions, scheduler, queue/usage policies, user quotas, container support, job preemption/uptime, simultaneous node availability and security boundaries.

Measure network topology and per-node NICs; test point-to-point bandwidth/latency, all-reduce and all-gather for realistic payload sizes, first within rack and then across racks. Record p50/p95 under competing load. Verify whether MPI/oneCCL/Gloo can use the fabric.

Test CPU tokenization/decompression, preprocessing, indexing and verifier throughput. These may deliver high immediate value independent of network.

Run full forward/backward/optimizer benchmark at 100M/300M/1B, 1K/2K/4K sequences, conservative microbatches and supported precision. Capture tokens/s, step-time distribution, resident memory, energy/thermal throttling, data starvation and checkpoint/restart. Repeat at least three times and a representative 24-hour job.

Test inference at batch 1 and parallel batch, prompt/decode split, quantization, context and failure recovery. Measure each node and fleet throughput.

Test model sharding only after collectives pass. Report scaling efficiency $S_n / (n S_1)$, memory per rank and end-to-end tokens/s; compare with a single-node baseline.

Capture storage read/write, checkpoint duration, shared-filesystem contention, scratch quotas and artifact backup.

Scale gate and recommendation

Current recommendation: **treat 70-84 systems as a potentially valuable CPU cluster whose first proven value will likely be data, verifiers, simulation, search, batch evaluation and small proxy work.** Use 100M-300M as the first training proxies; profile before 1B. Do not claim 7B training is ruled out by aggregate RAM alone, but do not promise it either: if nodes can be joined, FSDP can shard state in principle; the compute scenarios show that useful training throughput, fabric and allocation determine whether it is practical. For substantial base training, accelerator access remains the recommended route. The cluster may still support a novel, useful research-loop/RL evaluation program without full-scale pretraining.

Unknown after the user's update: exact model/SKU count, whether the 70-84 hosts share 32 GB/i9/UHD 770 specs, OS homogeneity, availability at the same time, network, scheduler, storage and benchmark throughput. Confirm each before final model-scale commitment.

Local Codex host distinction

The Codex host separately reported Apple M3 integrated GPU (10 cores) via `system_profiler`; CPU/RAM `sysctl` queries were restricted. It is not one of the reported VIT machines and was not included in node or throughput estimates.

Scaling and compute analysis

Project research document · source path: research/scaling/scaling-and-compute-analysis.md

Scaling and compute analysis

Scope and assumptions

The user reports approximately 70–84 VIT machine-learning-lab systems, each described as having a 13th-generation Intel Core i9, 32 GB RAM and Intel UHD Graphics 770. If all are simultaneously accessible and consistent, this corresponds to 2.24–2.688 TB of aggregate installed RAM across separate hosts, not one shared memory pool. Exact count/SKUs, per-node consistency, network, scheduler, storage, access policy, uptime and measured cluster throughput are unverified. The system may be joinable as a CPU cluster; this is a hypothesis to test, not a confirmed capability. All throughput values below are transparent planning scenarios, not measurements. Dense training first-order compute is $F \approx 6 \times N \times D$ (parameters $\times$ training tokens $\times$ 6 FLOPs), excluding attention overhead nuances, optimizer updates, failed runs, post-training and data/tool costs. It is an order-of-magnitude comparison, not a performance model. Chinchilla’s reported optimum is not a universal ratio; modern repeated-token training, quality, inference demand and data limits shift it.

Training compute scenarios

Model	Illustrative tokens	6ND FLOPs	Approx. wall time at 1 PFLOP/s sustained	100 TFLOP/s	10 TFLOP/s
Dense 7B	140B (20 tok/param)	5.88×10^{21}	68 days	1.87 years	18.7 years
Dense 14B	280B	2.35×10^{22}	272 days	7.46 years	74.6 years
Dense 32B	640B	1.23×10^{23}	3.90 years	39.0 years	390 years
Dense 70B	1.4T	5.88×10^{23}	18.7 years	187 years	1,870 years
MoE 100B total / 20B active	400B	4.80×10^{22} approximate active FLOPs	1.52 years	15.2 years	152 years
MoE 300B total / 32B active	1T	1.92×10^{23} approximate active FLOPs	6.09 years	60.9 years	609 years

At fixed sustained throughput, time scales linearly with FLOPs. 1 PFLOP/s sustained is a powerful accelerator-cluster scenario, not an i9 estimate. At 1 TFLOP/s per node, 1 PFLOP/s requires 1,000 nodes *at sustained useful training throughput*; at 100 GFLOP/s/node it requires 10,000. A CPU node’s actual transformer throughput depends on SIMD/precision support, memory bandwidth, kernel, sequence/batch and thermal/power scheduling. Measure it; theoretical peak is not sustained useful training throughput.

Token sensitivity is linear: halve/double tokens, halve/double estimated FLOPs. If quality corpora support only 50B unique tokens, a 7B model at 50B tokens costs 2.1×10^{21} FLOPs (about 24 days at 1 PF/s) but is 7 tokens/parameter. Repeating data may be useful but has diminishing returns and overfit risk; do not pretend it equals fresh tokens.

Memory estimates

BF16 weights: $2N$ bytes (7B $\approx$ 14 GB, 14B $\approx$ 28 GB, 32B $\approx$ 64 GB, 70B $\approx$ 140 GB; 100B–200B MoE $\approx$ 200–400 GB, 300B–400B $\approx$ 600–800 GB). This excludes buffers, fragmentation and quantization metadata.

Adam-style mixed-precision training commonly requires roughly 12–18 bytes/parameter for weights, gradients, FP32 master copy and two FP32 moments depending on implementation/state dtypes. Use a conservative 16 B/parameter planning point: 7B $\approx$ 112 GB; 14B $\approx$ 224 GB; 32B $\approx$ 512 GB; 70B $\approx$ 1.12 TB; 300B MoE $\approx$ 4.8 TB. Sharding distributes state but does not remove total bytes; activations, attention workspace, communication buffers and checkpoint staging are additional. MoE full weights/states follow total parameters; compute approximation follows active parameters plus routing/attention/shared layers.

KV cache per sequence for standard K/V is roughly $2 \times \text{layers} \times \text{seq_len} \times n_kv_heads \times \text{head_dim} \times \text{bytes}$, excluding allocator overhead. For example, 32 layers, 8 KV heads, head dimension 128, BF16, 32K tokens gives about 4 GiB per sequence. GQA reduces this proportional to KV heads; MLA changes formula, so profile implementation. Longer context can dominate serving memory even where weights fit.

Checkpoint size: BF16 model weights $2N$ bytes; full optimizer checkpoint can be $\sim$ 16N bytes before compression and metadata. Keep at least one immutable base, resumable latest, and milestone checkpoints; asynchronous save and quota must be measured. A 300B model’s BF16 weights alone are 600GB; full optimizer state is about 4.8TB. Replicas/retention multiply requirements.

Hardware decision thresholds

Measure sustained end-to-end tokens/s and FLOPs/s at intended context, not GEMM peak only. Include dataloading, checkpointing, validation, outages and utilization. Budget project time with effective throughput $P_{\text{eff}} = P_{\text{peak}} \times \text{utilization} \times \text{kernel efficiency} \times \text{availability}$; each factor should be measured or explicitly scenario-bounded. Estimate compute with 0.2/0.4/0.6 utilization cases until profiling supplies data. Inter-node collectives must be benchmarked for all-reduce and MoE all-to-all; count bytes/rank and topology oversubscription.

Recommendation

No final scale is authorized yet. Treat the reported 70–84-node fleet as a potentially useful CPU cluster, subject to access and benchmark gates. Start with 100M–300M single-node CPU proxies and parallelize data, verifiers, indexing, simulation, evaluation and rollouts across available nodes. Profile single-node and multi-node 1B runs; then decide whether a sharded 7B pilot has meaningful wall-clock value. Aggregate RAM could accommodate sharded state in principle (not as one pool), but does not establish practical training throughput. A 7B full run is conditional on measured end-to-end performance, network, schedule, data and budget; accelerators remain the preferred route for substantial pretraining. Consider 14B/32B and MoE only after scaling curves and full lifecycle resources justify them. Gate decisions on the audit in `../hardware/vit-ml-lab-audit.md`.

State-of-the-art architecture review

Project research document · source path: research/architecture/state-of-the-art-architecture-review.md

State-of-the-art architecture review

Decision context

The objective is an auditable scientific workflow, not an ungrounded “scientist model.” The literature supports decoder-only Transformers as the lowest-risk base family; MoE is an option when accelerator memory and network can support sparse routing. Alternatives such as state-space/hybrid models are worth benchmarked small-scale experiments but not a first-system dependency because tooling and transfer evidence for research workflows are less mature than the Transformer ecosystem.

Alternatives

Option	Advantages	Costs / failure modes	AIM decision
Dense decoder, shared researcher and decision representations	Simplest training, serving and debugging; one model can emit text and structured JSON; mature kernels/checkpoints.	All weights active each token; multiobjective gradient conflict; verbal confidence may be poorly calibrated.	Required 7B-class reference when compute exists; small proxy now. Add a trained decision head only as controlled variant.
Single backbone + dedicated typed decision head	Shared knowledge; low extra inference latency; schema-constrained outputs.	Head inherits representation errors; calibration can be domain-shifted; generation objective may conflict with decision scores.	Ablation after baseline; score head separately with proper loss.
Researcher + separate calibrated judge	Independent objective, data and update cadence; can score claims/route actions without rewriting prose.	Extra serving cost; judge can share blind spots; disagreement and correlated errors; stale-model coordination.	Best first integrated architecture, using small judge and verifier ensemble; require independent held-out correctness.
Researcher MoE + judge + deterministic verifier/controller	Sparse active compute can expand capacity; clear role boundaries; objective tools can certify narrow outputs.	Full expert memory and optimizer state; expert all-to-all; routing collapse/imbalance; operational complexity.	Future GPU research path only after 7B dense and a measured MoE proxy. No domain-pinned experts initially.
Retrieval-augmented existing model, no new pretraining	Fastest useful system, fresh evidence and lower compute; tests whether loop/value comes from tooling.	Capability ceiling and license/availability dependence; no base-model contribution.	Must be a systems baseline. Research novelty can be workflow/training even if initial researcher is a licensed open model.

Proposed model family, conditional on accelerator access

Use a conventional pre-norm decoder Transformer, RMSNorm, SwiGLU MLP, RoPE, GQA and BF16 (or hardware-supported FP8 after validated numerics). These are evidence-backed engineering defaults, not a claim they are uniquely optimal. Begin with 32 layers, hidden dimension 4096, 32 query heads, 8 KV heads, head dimension 128, and SwiGLU intermediate width chosen to land near 7B parameters; parameter count must be computed from the actual tokenizer vocab, embedding tying, bias policy, and exact layer spec before run configuration. A 7B-equivalent spec is an experiment target, not selected VIT scale. 14B/32B/70B require separate budget and scaling evidence.

Context plan: train robustly at 4K–8K first, extend to 16K–32K only with long-sequence data and memory profiling, and serve longer evidence through retrieval/chunking rather than forcing every request into a maximum context. Use RoPE base/position settings with length curriculum; test native training versus YaRN-style extension on retrieval, positional extrapolation, and long-document synthesis. GQA reduces KV heads/cache but is not free: memory scales approximately with layers × sequence × KV heads × head dimension × bytes × 2 for K and V per request.

Tokenizer: compare a modern byte-fallback BPE/unigram tokenizer trained on licensed, domain-balanced text. Include scientific notation, Unicode math, LaTeX, code, tables, biomedical terms and multilingual samples. Evaluate compression by domain, round-trip loss for symbols, segmentation of rare equations, and tokenization fairness; freeze tokenizer before expensive pretraining. Avoid claiming a tokenizer is universally optimal.

MoE decision: only consider after dense scaling proxy says capacity is limiting rather than data, data quality or post-training. Prototype modest 8–16 experts/layer, top-2 routing with capacity and explicit load-balance monitoring; compare Switch-style coarse experts with fine-grained/shared-expert DeepSeekMoE design at equal active FLOPs. Keep experts data-driven; evaluate specialization and cross-domain transfer before domain labels are used in routing. MoE uses active parameters for approximate per-token dense FLOPs but *total* expert parameters for weight residency, optimizer and checkpoint storage; token dispatch requires all-to-all. Dense is preferred on weak/intermittent networks.

Position and attention: RoPE is a strong, portable baseline; GQA is a cache optimization with quality tradeoffs to measure. FlashAttention-2 is exact attention with IO-aware kernels, but only where supported by the device stack. MLA can reduce KV memory, as reported by DeepSeek-V2/V3, but adds architectural and kernel complexity; benchmark at matched quality before considering. Do not assume NoPE, linear attention, state-space or hybrid block designs beat a plain Transformer for AIM: benchmark one representative alternative only if the dense control is bottlenecked by sequence or memory.

Objective/hardware: next-token cross entropy on packed documents with document boundaries and loss masks; AdamW baseline; compare Muon on a small preregistered scaling slice. BF16 baseline where accelerator supports it. Use activation checkpointing, fused kernels, async data prefetch, and sharded optimizer/model state when GPU distributed training is selected. Choose TP/PP/DP/EP from topology measurements; high-latency Ethernet is not equivalent to NVLink/InfiniBand.

Evidence and tradeoffs

DeepSeek-V3 is a concrete sparse reference (671B total, 37B activated, 14.8T training tokens) but trained on dedicated H800 infrastructure; it demonstrates feasibility for a well-resourced organization, not feasibility on the reported VIT CPU fleet absent measured throughput and interconnect benchmarks. Qwen3 spans dense and MoE and supports thinking/non-thinking modes. GLM-4.5 reports 355B total / 32B active. These reports support architectural plausibility, not an AIM scale choice.

DeepSeekMoE's expert splitting/shared experts show one viable load/specialization technique. Muon reports improved compute efficiency in its own experiments; AdamW remains the conservative control.

Parameter-scale gate

Measure tokens/s, sustained training FLOPs/s, memory headroom, all-reduce and all-to-all first. Then fit loss-vs-compute scaling proxies across at least three model sizes and several token budgets. If the funded compute cannot complete a credible 7B training run on licensed tokens, treat 7B as a target only if accelerator access is secured; do not train a nominal 32B underfit artifact for marketing. The immediate low-risk build is retrieval/system prototype plus 100M–300M single-node CPU proxies, while using the reported fleet for parallel data/verifier/evaluation workloads. A distributed 1B pilot is a benchmark gate, not a presumed outcome. For funding: solicit a GPU allocation or cloud grant after reproducible throughput data exists.

Tests that could change this recommendation

Dense versus MoE proxy at matched active FLOPs and data: held-out loss, domain transfer, load balance, wall-clock, energy and serving cost.

GQA/MLA variants at equal training compute: long-context accuracy and cache bytes/token.

Native long-context training versus retrieval: evidence recall and supported synthesis at same GPU-hours.

Shared decision head versus separate judge: Brier/log score and selective risk at fixed cost.

AdamW versus Muon: tokens-to-target validation loss, sensitivity and restart reproducibility.

RLHF, RLVR, RLCD review

Project research document · source path: research/rl/rlhf-rlvr-rlcd-review.md

RLHF + RLVR + RLCD: review and proposed integration

Main conclusion

These labels describe unlike supervision. RLHF targets human-valued quality/preferences; RLVR targets task outcomes that a trusted program can check; RLCD, as publicly described by TypeSafe in September 2026, targets structured probabilistic decisions. A single weighted scalar can obscure failures and let a high prose-preference score compensate for an incorrect result. AIM should begin with separated datasets, heads/policies where useful, and constrained gates; only combine rewards after measuring scale, noise, and tradeoffs.

The Jev/RLCD announcement is a company claim. Its public materials describe typed decisions and a probability distribution/confidence; the “zero hallucination” wording is expressly about output type/range conformance. The public material reviewed does not disclose a reproducible loss, RL algorithm, training data, or independent calibration results. Thus RLCD here means an AIM research objective family, not an implementation copied from Jev.

Mathematical objects

Autoregressive pretraining minimizes token negative log-likelihood: $\mathcal{L}_{LM}(\theta) = -\mathbb{E}_{x \sim D} \sum_t m_t \log \pi_\theta(x_t | x_{<t})$, where m_t masks padding or excluded spans. This is not a direct factuality objective.

For pairwise preferences, Bradley–Terry models $P(y_w \succ y_l | x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$, optimized with binary cross entropy. The conventional KL-regularized target is $\max_\pi \mathbb{E}[r(x, y)] - \beta \text{KL}(\pi || \pi_{\text{ref}})$. PPO clips the probability ratio $p_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\text{old}}(a_t | s_t)$ in the surrogate $\mathbb{E}[\min(p_t A_t, \text{clip}(p_t, 1-\epsilon, 1+\epsilon)A_t)]$, with value/GAE and KL controls in common implementations (PPO). RLOO is a REINFORCE-style leave-one-out baseline that can avoid a separate value/critic model (Ahmadian et al.); compare it where critic memory is material. DPO eliminates the separate online RL loop under its assumptions; it is an excellent low-cost preference baseline, not proof that RL is unnecessary for sequential environment interaction.

Preference-algorithm comparison for AIM: **DPO** is a clean pairwise/reference-policy baseline; **IPO** changes the preference objective to address issues in DPO's assumptions and should be evaluated where label noise/ties matter (Azar et al.); **KTO** uses binary desirable/undesirable feedback and a prospect-theoretic utility, useful when pairwise labeling is expensive but it changes assumptions about human utility (Ethayarajh et al.); **ORPO** integrates an odds-ratio penalty into SFT and removes a separate reference model, reducing orchestration but coupling instruction imitation and preference learning (Hong et al.); **SimPO** uses length-normalized sequence log probability as an implicit reward with a target margin, avoiding a reference model (Meng et al.). **PPO-RLHF** enables on-policy exploration through an explicit learned RM but costs rollout/critic infrastructure and is vulnerable to RM overoptimization. **RLAIF/Constitutional AI** can scale critique/preferences but inherits evaluator bias and should be validated against human/domain labels (Bai et al.). **Self-rewarding** systems risk circular teacher/student errors; use only with external objective checks. AIM should implement DPO first, then one alternative at a time on the same preference set; select by blind research utility and factuality, not headline chat leaderboard.

For RLVR, an episode τ receives a terminal or verifiable reward such as $r_v(\tau) \in \{0, 1\}$ from a grader, theorem prover, tests, or recomputation. REINFORCE uses $(r - b) \log \pi(\tau)$. PPO applies clipped token/trajectory ratios. GRPO samples a group for a prompt and centers/scales outcome rewards, often $A_i = (r_i - \text{mean}(r)) / (\text{std}(r) + \epsilon)$; this removes a learned value model but creates group-relative and difficulty/length effects. DAPO changes sampling/clipping/normalization choices; Dr. GRPO analyzes normalization bias. These are algorithm variants to ablate, not guaranteed improvements.

For pairwise choice probabilities p , Brier loss is $(p - y)^2$ (multiclass $\sum_k (p_k - y_k)^2$) and log loss is $-y \log p - (1 - y) \log(1 - p)$. They are strictly proper under standard assumptions: expected loss is optimized by reporting the true conditional distribution. Brier is bounded but decomposes discrimination and reliability; log loss punishes confident errors strongly. Use both. ECE bins confidence and compares mean confidence/accuracy, but is bin-dependent, sample-hungry and not a proper scoring rule; report confidence intervals, adaptive bins, reliability curves and proper scores. Accuracy alone cannot measure calibration.

Selective prediction with action `abstain` and cost c_a should be trained/evaluated against actual operational costs. For a calibrated binary event with action loss $L(a, y)$, choose $a^* = \argmin_a \mathbb{E}[L(a, Y) | x]$; abstain when its expected loss is lower. Thresholds are product costs, not a universal magic confidence. Track risk versus coverage, utility under preregistered costs, and whether abstention is useful (not blanket refusal).

Why scalar reward is risky

Suppose outcome quality has vector $R = (R_h, R_v, R_c, R_p)$: human usefulness, verified correctness, calibration/decision quality and process quality. Scalarization wDR chooses a tradeoff only if scales and weights represent real exchange rates. If $R_v = 0$ (fatal fact/error) but R_h is high, unconstrained sum may still reward it. Prefer a hierarchy / constraints for critical correctness: $\lfloor \max_{\pi} \pi [E[R_h + \lambda R_{\text{progress}}] - \beta KL(\pi || \pi_{\text{ref}})] \rfloor \text{quad} \text{t} \{s.t. \} E[1 - R_v] \leq \epsilon_v, \lfloor E[\text{unsupported claim rate}] \rfloor \leq \epsilon_u, \rfloor$ with calibration optimized and reported separately. If constraint optimization is unstable, use lexicographic selection: verifier pass $\rightarrow$ supported claims $\rightarrow$ calibrated control decisions $\rightarrow$ preference quality, with a Pareto dashboard. The calibration score applies to decisions with observable outcomes, not to all prose assertions at once.

Proposed staged experiment, not final recipe

Research SFT: train output/action formats and correct tool use from curated demonstrations. Keep raw sources and provenance; do not label generated chain-of-thought as ground truth by default.

Preference baseline: DPO on human pairwise comparisons over answer usefulness, clarity, relevance, and faithful reporting. Maintain separate dimensions and expert disagreement; sample for human adjudication. Compare PPO+learned RM only if online interaction/data justify complexity.

RLVR islands: separate math, code, symbolic and simulation environments with trusted execution, hidden tests, timeouts, sandboxing and independent test generation. Use outcome reward first. Compare PPO vs GRPO at same sample and accelerator budget.

Calibrated decision policy: take immutable research-state snapshots; emit typed distributions for claim state, verify/continue/retrieve/abstain and candidate action ranking. Train with proper scoring losses on adjudicated outcomes and logged environment returns. Start offline supervised calibration; test RL only where action consequences yield clean delayed rewards. Keep decision calibration metrics outside language-policy scalar reward initially.

Process signal: evaluate PRM as candidate-step ranking / critique auxiliary trained on step labels and outcomes. It must not directly certify truth. Use outcome-conditioned process targets, counterfactual invalid-step pairs, and terminal verification; compare against no-PRM.

Integrated research-loop RL: state/action sequence s_t, a_t, o_{t+1} , rewards for verified evidence, successful experiments, supported final claims and cost. Use deterministic environment contracts. Run factorial ablations: no RL; DPO only; RLVR only; decision policy only; combinations; process reward; shared vs separate judge.

Delayed and unobservable truth

For scientific claims with no immediate ground truth, do not fabricate labels by forcing a binary truth score. Calibrate only an explicitly defined target, horizon and reference class, e.g., “does this claim follow from these provided sources?” versus “will this hypothesis replicate in a future experiment?” The former has a present, auditable label; the latter may require longitudinal outcomes and censoring. Store forecast issue time, resolution rule, observation horizon, and unresolved status. Use proper scoring only after outcome resolution; treat missing resolution as censored, not incorrect or correct. For causal/hypothesis novelty, human expert review and future studies are needed.

Adversarial answers

Does the trio improve research ability? **Unknown**; only controlled ablations answer it.

Calibration without outcomes? Only proxy/reference-class calibration; cannot empirically establish unknown future truth.

Delayed outcomes? Forecast ledger, fixed resolution criteria, time-to-event/censoring, periodic follow-up.

Can judge supervise generator? Yes for action choice/selection if labels are valid, but it cannot prove unseen prose true; maintain independent auditing.

Judge abstains when expected decision cost exceeds abstention cost, out-of-support input, conflicting verifiers, missing provenance, or low effective sample support.

Prevent confidently wrong judge: held-out domains/time periods, independent judges/verifiers, adversarial positives/negatives, calibration confidence intervals, distribution-shift detector, human audit.

Stop verifier gaming: hidden tests, metamorphic/property tests, randomization, sandbox, exploit red-team, verifier versioning and audits.

Prevent verbosity reward: length-normalized preference comparisons, claim-level utility, cost penalty, blinded length controls, no reward for visible reasoning volume.

Process reward may guide search but risks rewarding plausible style; test on novel tasks and wrong-but-plausible traces. Include the outcome-guided process-reward study cited in the Directive ([Rezaei et al., 2026](#)) alongside PRM800K; verify scope and reproduce before adopting.

MoE fragmentation: not established as a general research risk/benefit; measure cross-domain transfer, route entropy, expert mutual information and ablation with same active compute.

11–13. Pretraining, data and scale bottleneck are open; estimate by nested ablations. Likely bottleneck is not simply parameters: high-quality independent feedback, verifiers and compute matter.

CPU clusters strongly help data, search, tests, synthesis, evaluation and CPU-native scientific jobs; contribution to full pretraining depends on measured sustained throughput and cost.

New contribution requires preregistered ablation showing improved trajectory-level, independently graded research outcomes per compute, robustly across held-out domains, beyond retrieval/tool and same-model baselines.

Reward and verifier design

Project research document · source path: research/rl/reward-and-verifier-design.md

Reward and verifier architecture

Principle

No evaluator is a universal truth oracle. Treat each verifier as an instrument with a declared input contract, target, error profile, version, and audit set. The final system should retain raw outputs and never convert “judge says supported” into “verified fact” without the declared evidence check.

Stack

Schema/contract checker: JSON schema, tool arguments, required claim IDs, references resolving to stored evidence. Deterministic and fail-closed.

Domain verifiers: code build/tests and complexity checks; symbolic algebra with assumptions; theorem proof in Lean/Isabelle where formalization exists; numerical simulation with fixed seed/tolerances; statistics recomputed from data; dimensional/unit consistency. Each verifier reports pass/fail/unknown plus logs.

Source checker: resolve bibliographic identity, archive source/version/hash, map claim to exact page/section/evidence span, test entailment and counterevidence. Entailment models can triage, not certify.

Independent judge models: compare claim-evidence pairs for support, contradiction, missing qualification; separate from researcher weights/data when possible. Calibrate on human-labeled hard negatives.

Human expert panel: adjudicate high-impact claims, ambiguous proofs, novelty and research quality; record disagreement rather than force consensus.

Calibrated controller: on explicit candidate action set, emit action probabilities and abstention/review trigger. Output contract prevents invalid types; correctness depends on calibration and coverage.

Reward signals

Maintain decomposed event ledger per trajectory: task success, independently verified claim fraction, citation entailment, experiment reproducibility, hypothesis discrimination, calibration proper score after resolution, human preference, tool cost, wall time, and policy violations. Use hard gates for invalid/malicious tool behavior and missing provenance. Use scalar reward only within an isolated, well-defined environment with reward scale/variance recorded. Use constrained/Pareto comparisons at system level.

Outcome reward is preferred for objective tasks. Process score may improve search or identify a bad step in a proof, but use as a proposal prior or bounded shaping term. A candidate step reward should be terminally checked and outcome-conditioned: $r_t = f(\text{step}_t, \text{context}, \text{eventual verified outcome})$, trained on correct and adversarially plausible traces. Do not pay per token or reward “reasoning-looking” text. Ensure any shaping preserves the desired optimal policy (e.g., potential-based $F(s, s') = \gamma \Phi(s') - \Phi(s)$ under its assumptions); arbitrary shaping can change it.

Anti-reward-hacking controls

- Isolate verifier inputs from model-writable state; tool results are signed/immutable and provenance-bound.
- Use sandboxed execution, no secret leakage, CPU/memory/time quotas, network disabled unless explicitly required, and fresh containers for reproduction.
- Keep hidden tests and test generators outside training access. Rotate held-out seeds and challenge sets.
- Use differential verification (two independent implementations, symbolic + numeric, reference + metamorphic properties), especially when a single parser can be exploited.
- Evaluate false-accept and false-reject rates separately with confidence intervals. Human-audit a random sample and all high-impact cases.
- Freeze reward versions per run; log input, version/hash, output and policy decision. Canary old/new judges to detect drift.
- Check reward against task outcomes under optimization pressure, not only IID classifier accuracy. Track proxy score vs independent score as policy moves away from reference.
- Penalize unnecessary tool calls and verbosity only after useful work is measured; otherwise incentives encourage premature stopping.

- Red-team answer leakage, test memorization, formatting tricks, citation laundering, and code that exploits harness state.

Verifier contracts and output states

VERIFIED means a specific declared check passed for a specific claim and input (not “scientifically true forever”).

STRONGLY_SUPPORTED means multiple relevant independent sources with adequate spans and no detected contradiction; this remains an evidence judgment. PARTIALLY_SUPPORTED indicates a narrower statement is entailed. UNCERTAIN, UNVERIFIED, CONTRADICTED, HYPOTHESIS, and UNKNOWN preserve epistemic distinctions. Every state includes method, timestamp, scope, and provenance. Confidence and verification state are separate fields.

For numerical/code facts store artifact hashes, environment/container, dependencies, command, seed, tolerances and stdout/stderr. For citations store DOI/URL, version date, retrieval timestamp, source hash, page/line/span offsets and quote hash. Keep extracted text alongside offsets and parser version.

Data strategy

Project research document · source path: research/data/data-strategy.md

Data strategy: quality, rights, and provenance

Policy

Publicly downloadable is not equivalent to licensed for model training. Store terms and license at asset/document level, plus jurisdiction, source URL, retrieval time, permitted use, attribution/share-alike/noncommercial restrictions, opt-out/takedown status, and legal review. Default unknown or incompatible rights to retrieval-only or exclude; do not scrape paywalled or access-controlled content. This is an engineering policy, not legal advice.

OpenAlex metadata is documented as CC0, but its license fields describe locations/copies of works and do not make each linked full text CC0. Semantic Scholar's dataset/API agreement carries use restrictions. arXiv papers have article-specific terms and repository policies; inspect each record. Code needs per-repository/per-file licensing and obligations; avoid assuming GitHub public means permissive. Maintain deletion lineage so a source can be removed and affected datasets/retrains identified.

Data layers

Layer	Content	Use / controls
A. Knowledge	CC/PD reference works, licensed educational sources, permitted OA papers, documentation, tables.	Deduplicate near copies, preserve bibliographic and temporal metadata, reduce boilerplate, balance fields/languages.
B. Structured reasoning	Math problems and proofs, code with tests, derivations, protocols, worked examples, formal proofs.	Confirm dataset licenses; preserve problem provenance; create train/validation/test by generator/source family.
C. Research process	Query → search → paper reading → extraction → competing hypotheses → experiments → results → conclusion.	Prefer observed, reproducible traces; store exact tool environment and evidence; review human examples.
D. Negative/corrective	Invalid proof, code bug, false citation, unsupported claim, confounded experiment, correction.	Pair error with detector and repair; label error type and severity; avoid teaching harmful fabrication as successful target.
E. Synthetic environments	Procedurally generated math, causal/scientific worlds, simulated experiments, API/tool worlds.	Ground truth and generator held out by family; randomize superficial templates; publish generator and seeds after evaluation where safe.

Construction pipeline

Ingest manifests first; enforce allowlist and rights gate before content fetch.

Normalize Unicode/layout while retaining original raw object and hash.

Detect PII/secrets, malware, spam, low-quality duplication, benchmark contamination and license conflicts.

Parse paper structure into abstract/sections/equations/tables/references with page/span offsets and parser confidence.

Deduplicate exact, near-duplicate and boilerplate text across splits; use source/group splits to prevent leakage.

Tag discipline, genre, date, language, evidence type, license, source reliability and quality; retain uncertain labels.

Build mixture experiments rather than assume fixed percentages. Tune using domain-held-out validation loss and downstream trajectory metrics.

Track token counts before and after filtering, source concentration, language/domain coverage, rights exclusions and estimated unique-token count.

Initial mixture hypothesis

Do not lock percentages until pilot data. Start with natural-language licensed knowledge plus high-quality textbooks/reference, software and scientific code, mathematics, scholarly papers, and process trajectories. Oversample high-value reasoning in mid-training rather than letting small scientific corpora dominate base pretraining. Use quality/domain weighting with caps per source and deduplication. Maintain a broad general-language anchor to avoid catastrophic forgetting. Validate exact token allocation by sweep.

Synthetic data

Generate trajectories from symbolic/math solvers, compilers/tests, theorem provers, simulators and controlled causal worlds. Record the generator executable/version, hidden seed, expected invariants, and independent checker. Sample success, near misses, irrelevant evidence, contradictory sources, broken tools, confounders and underdetermined cases. Prevent train/test family leakage by holding out generator algorithms and latent rules, not just random seeds. Model-generated explanations may be included only when independently validated; teacher confidence is not ground truth.

Rights tiers

- **Train-ok pending legal review:** explicit permissive license covering intended use, compliance recorded.
- **Conditional:** share-alike, attribution, noncommercial or jurisdictional constraints; do not mix into a model intended for incompatible redistribution until counsel approves.
- **Metadata-only / retrieval-only:** metadata terms permit indexing but full-text/model-training terms are unclear or restricted.
- **Excluded:** unknown, access-controlled, paywalled without license, opt-out, or incompatible terms.

Data lineage must reach final token chunks and checkpoint/run IDs. Publish a datasheet listing categories, counts, filtering and known gaps without disclosing restricted content.

Agentic research-loop design

Project research document · source path: research/agentic-research/research-loop-design.md

Research-loop design

Architecture choice

Use a hybrid deterministic shell + learned researcher + explicit decision model + verifier tools. Rules enforce permissions, provenance, schema and stop conditions; learned policies choose what to read, compare, calculate, test or ask next. An entirely rule-based workflow is brittle; an unconstrained agent can silently lose state and provenance. Compare the proposed controller with (1) one LLM acting from transcript, (2) LLM with structured state but no learned router, and (3) separate planner/judge. Evaluate at equal token/tool budget.

Typed research state

```
ResearchState = {
  question: Question {text, scope, time_cutoff, requested_artifact},
  assumptions: [Assumption {id, text, status, source}],
  known_facts: [ClaimRef], unknowns: [QuestionRef],
  sources: [Source {id, url_or_doi, version, retrieved_at, license, hash}],
  evidence: [EvidenceSpan {source_id, locator, text_hash, parser, relevance}],
  hypotheses: [Hypothesis {id, statement, predictions, alternatives}],
  experiments: [Experiment {id, protocol, preregistration, code_hash, seed}],
  results: [Result {experiment_id, raw_artifact, status, uncertainty}],
  contradictions: [Relation {claim_a, claim_b, evidence_ids, resolution}],
  confidence: [Forecast {claim_id, target, probability, reference_class, horizon}],
  open_tasks: [Task {action, rationale, expected_information_gain, cost}],
  provenance: [DerivationEdge], final_claims: [Claim {text, epistemic_state, evidence_ids}]
}
```

State is append-only event history with materialized current view. Every tool result carries immutable event ID, time, environment/version and source references. Edits are new events with supersedes/retracts links. This supports “why believe it?”, contradictions and belief revision. Do not use a vector index as sole memory.

Action protocol

Actions: SEARCH, READ, EXTRACT, COMPARE, CALCULATE, CODE, RUN, SIMULATE, PROVE, CHALLENGE, HYPOTHESIZE, VERIFY, ABSTAIN, REVISE, WRITE. Each has JSON schema, capability scope, cost limit, timeout, expected observation schema, and idempotency policy. The system must distinguish failure, empty result, timeout, unsupported and verified false. Execute code in a disposable sandbox; Internet access only through explicit retrieval tools; no hidden filesystem/secret access.

Agent cycle

Normalize question, scope, target and deadline; surface ambiguities as explicit assumptions.

Build claim graph and research plan with competing explanations.

Search primary/credible sources; capture exact version and spans, not just snippets.

Extract claims/method/data/limits separately; cross-check source identity and dates.

Generate at least two plausible hypotheses where the task calls for discovery; derive distinguishing predictions.

Choose next action by expected value of information (EVI), risk and cost; controller abstains/asks for human input when environment lacks needed authority or evidence.

Run independent checks/experiments; record raw outputs.

Update claims and forecasts; preserve negative results and unresolved conflicts.

Draft only from claim graph; automated citation coverage check; human review gates for high-impact assertions.

Stop when preregistered evidence/utility criteria met, budget exhausted, or marginal EVI below cost; report what remains unknown.

Memory architecture

Use relational tables for source/version/claim/evidence/experiment/provenance and constraints, object storage for PDFs/code/output artifacts, graph edges for support/refute/derived-from/cites/supersedes, and a vector index for candidate retrieval only. Add lexical BM25 and citation/metadata search; rerank with source quality and date filters. Store embeddings with model/version. Retrieval returns passages with locators and rights boundaries. Version source changes; never silently overwrite. Use user/project workspaces with access controls and retention/deletion policy.

Training loop as sequential decision process

State s_t includes bounded evidence graph summary plus durable database IDs; action a_t is a typed research operation; observation o_{t+1} is retrieved/verified result; terminal report y . Reward components include verified task progress, supported claim coverage, experiment reproducibility, resolved uncertainty, preference quality, and resource cost. Credit assignment should use terminal outcome and intermediate *observable progress* (e.g., newly verified relevant evidence), not verbose intermediate rationale. Train trajectories offline from reproducible tasks, then cautious online policy optimization in sandboxed environments. Randomize tool failures/data order and hold out entire environment families.

Novelty gate

“Research-Loop Reinforcement Learning” is not established as novel by naming it. Define contribution as a measurable policy-learning method over evidence-state actions with clean environment reward and superior independent research outcomes at fixed compute/cost versus ReAct/tool-using SFT, retrieval+same model, and a fixed planner. Require ablation for learned decision policy, provenance state, verifier ensemble and reward components. Preregister before test evaluation.

Evaluation and benchmark plan

Project research document · source path: research/evaluation/evaluation-and-benchmark-plan.md

Evaluation and benchmark plan

Principles

Build internal held-out tasks before training. Public benchmarks are diagnostic, not proof of research ability. Separate base knowledge, post-training, retrieval, tool policy, verifier, calibration and system-level effects. Test contamination by source- and time-based splits; preserve a sealed set and third-party red-team items. Report paired runs, confidence intervals, sample size, inference budget and tool cost. Never compare models with different access/budget without disclosure.

Matrix

Capability	Measures	Task design
Knowledge	Accuracy, temporal error, calibration, source support	Domain-stratified factual Qs; date cutoffs; conflict/false premise; with/without retrieval.
Mathematics	Exact correctness, proof validity, verifier pass, compute/calls	MATH-style and expert-held-out tasks, symbolic equivalence, Lean proof, novel generator families, partial credit by proof rubric. FrontierMath access/use only under terms; avoid relying on exposed answers.
Code	Hidden-test pass, compile, runtime, complexity, security	SWE-bench/Verified where licensed and reproducible, internal issue tasks, property tests; fresh container reproduction.
Science	Hypothesis discrimination, experiment validity, power/confound checks	Synthetic causal worlds plus expert-authored novel scenarios; score predictions against held-out environment outcomes.
Literature	Supported-claim precision/recall, citation entailment, contradiction recall, source quality	ALCE-style attributed synthesis with exact spans; versioned papers; include contradictory/withdrawn evidence.
Research replication	Reproduction rubric, result closeness, artifact quality, cost	PaperBench-style separated agent/reproduction/judge containers; add non-ML and negative-results tasks.
Calibration	Brier, log score, reliability plots, ECE as secondary, selective risk–coverage	Correctness for clearly defined claims, decision/action outcomes, source-support judgments; report each separately and by shift.
Long-horizon	Verified evidence gained, claim traceability, missed contradictions, budget efficiency	10–50-step investigation; perturb order, inject tool outages and conflicting sources; audit event log.
Writing	Blind expert preference, factual precision, citation completeness, compression	Pairwise rating; length controlled; judge disagreement reported.
Domain breadth	All above by math, CS, physics, quantum, astronomy, chemistry, biology, psych/cog sci, statistics, engineering, economics, philosophy/logic, methods and scholarly writing	Report per-domain intervals; no equal-mastery claim from pooled score.

Calibration details

Define event γ before asking a probability: e.g., “Does passage S entail claim C under the stated scope?”, “Will hidden tests pass?” or “Will experiment outcome fall in interval by date?”. A system forecast about future scientific truth needs horizon and resolution rule. Score resolved events with mean Brier and log loss; report calibration intercept/slope and stratified reliability. ECE is descriptive and bin-sensitive, not the sole optimization/evaluation target. For multiple choices use multiclass proper score. For claims with a truth spectrum, use ordinal/distribution targets or expert rubric, not arbitrary binary collapse.

For abstention, show risk vs coverage, area under risk-coverage curve, utility at preregistered review costs, and confidence intervals. Count abstentions on answerable tasks and confidently wrong answers separately. Verify calibration under domain shift, source removal, temporal shift and tool failure.

Judge governance

LLM-as-judge is not ground truth. Create human adjudication subset with ≥ 2 independent domain reviewers and disagreement adjudication; validate judge against it, blind model identities, rotate rubric order and test verbosity/position bias. Report judge sensitivity/specificity and subgroup performance. For objective verifiers, independently audit grader false passes/fails and property-test edge cases.

Ablations and gates

Baseline ladder: base model; SFT; retrieval+SFT; tool loop; DPO; RLVR; calibrated judge; research-state memory; process reward; integrated controller. Factorially remove each component. Test fixed token/GPU-hour and fixed tool-call budgets. A component graduates only if it improves predeclared primary trajectory metric without violating safety/factuality/calibration guardrails. Repeat at least 3 seeds for small runs and report all; larger runs with fewer seeds need paired tasks and explicit uncertainty.

Primary system endpoint: **independently verified supported research progress per unit cost**, decomposed into (a) correct resolved subquestions, (b) claim-level citation support, (c) falsified hypotheses correctly rejected, (d) reproducibility, and (e) unresolved question reporting. This is a proposed operational metric; validate that it reflects expert judgment before optimizing it.

Open research questions

Project research document · source path: research/open-questions/open-research-questions.md

Open research questions and falsification plan

Question / hypothesis	Current status	Minimum experiment that can falsify it	Decision consequence
RLHF + RLVR + calibrated decisions improve research outcomes jointly.	Hypothesis, not demonstrated for AIM.	2×2×2 factorial post-training on same 1B proxy/data; sealed multi-step tasks; fixed token/tool budget, 3 seeds.	Drop any component without independent gain or if it harms core metrics.
Calibrated decision model is better separate than a head.	Unknown.	Shared head vs separate judge with same training examples and inference budget; domain/time-shift Brier, log score, selective utility.	Choose simplest configuration on Pareto frontier.
Structured provenance state reduces unsupported claims and forgetting.	Evidence-backed motivation; AIM effect unknown.	Transcript-only vs append-only evidence graph on matched long tasks; blind adjudication and injected contradiction.	Build only if traceability and success gains outweigh latency.
Process reward helps beyond outcome verification.	Mixed/narrow evidence.	PRM vs no PRM on proof and simulation environments; hold out step patterns; probe plausible wrong steps and verbosity.	Keep only bounded search/critique role if robust.
MoE improves quality per FLOP without harmful domain fragmentation.	Plausible, not shown for research.	Dense and modest MoE proxy, same active FLOPs/tokens; transfer, expert utilization, all-to-all, wall-clock.	Require clear operational advantage before scale-up.
Native long context is preferable to retrieval/chunking.	Unknown and workload-dependent.	Matched evidence tasks, context lengths, retrieval; score exact span recall, synthesis, cost and latency.	Train longer only where measured net improvement.
Better pretraining data beats larger parameter count at project budget.	Strong scaling-law motivation; project-specific unknown.	Nested proxy runs with quality-mixture vs broad corpus at equal compute.	Use loss+research task Pareto to allocate budget.
CPU cluster can deliver more than infrastructure support.	Likely for preprocessing/verifiers; throughput unknown.	Audit plus 24h proxy benchmarks and job cost accounting.	Decide local jobs vs accelerator request/cloud by workload.
System learns to update beliefs rather than rationalize initial answer.	Hypothesis.	Hidden sequential evidence tasks with randomized order, contradiction injection, scoring for appropriate revision and preserved history.	Make belief revision primary outcome if successful.
Research-loop RL is a novel contribution.	Speculative.	Compare learned policy to strong fixed/retrieval baselines; independent replication and pre-registration.	Claim novelty only if causal gains generalize and are not scaffold artifacts.

Unresolved empirical and governance questions

- What accelerator type, quantity and reserved hours are actually available at VIT? What are cluster access and data rules?
- How many unique, explicitly reusable tokens can be assembled across scientific and educational domains?
- Which primary endpoint do domain experts consider useful and reliable enough to guide research-system development?
- Can support/calibration labels be collected consistently across disciplines, and how should disagreement be represented?
- How much calibration transfers between a model's own claims, claims grounded in supplied sources, and future outcomes?
- What is the response to source retraction, version correction, and evidence graph deletion?
- What security envelope is needed before tool actions can run code or access network resources?
- Which benchmark families remain unexposed and licensed for public evaluation?
- Can model-assisted grading be calibrated sufficiently for large-scale science-task rubrics, or is expert review the bottleneck?
- What is a realistic objective for novelty? Novelty detectors are retrieval heuristics, not proof of new knowledge.

Evidence needed before publication

Preregistered hypotheses and exclusions; benchmark/data rights; sealed test suite; runs at matched budget; independent verifier audit; baseline and ablations; negative results; confidence intervals; reproducible configs/environment; external replication; narrowly worded claims. “Research intelligence” or “scientific discovery” must be supported by trajectory outcomes, not a demo or benchmark composite alone.

Architecture specification

Project research document · source path: documents/final-design/AIM-Research-Intelligence-Model-Architecture-Spec.md

AIM Research Intelligence Model — Architecture Specification

Version: 0.1 research proposal, 24 September 2026 **Decision status:** conditional; preliminary workstation specification supplied, full VIT inventory and benchmarks still pending. **Source documents:** Astra Master Research Directive v0.1 and Research Intelligence Model Charter v0.1, treated as project vision and hypotheses.

1. Proposed system

AIM is a system composed of (1) an autoregressive Researcher that proposes claims, hypotheses, derivations and actions; (2) a typed decision/judge component for route/continue/retry/abstain and evidence-status decisions; (3) deterministic and learned verifier ensemble; (4) a controller that maintains append-only research state; (5) provenance/evidence storage and retrieval; and (6) human review for high-impact or unresolvable claims.

Recommendation is **Variant C at the system boundary, not necessarily a giant MoE neural network**: researcher + separate small judge + deterministic shell/verifiers. Compare shared-head Variant A and researcher+judge Variant B. Choose the simplest system that wins trajectory ablations.

2. Model specification and unresolved scale

Current scale decision

The user reports approximately 70–84 VIT machine-learning-lab systems, each described as a 13th-generation Intel Core i9 with 32 GB RAM and Intel UHD Graphics 770. This remains preliminary inventory, not an independent audit or benchmark. The conditional aggregate is 2.24–2.688 TB RAM across separate hosts, not a shared pool. Exact node consistency/SKUs, network, scheduler, storage, access and throughput are unknown; whether this can be operated as a cluster must be demonstrated. Begin with 100M–300M single-node CPU proxies and parallel cluster workloads; benchmark multi-node scaling at 1B and evaluate a sharded 7B pilot only if measured throughput supports a useful schedule. Distributed sharding may address model-state capacity, but communication and CPU compute may dominate. No final parameter count exists today. Accelerator access is preferred for substantial pretraining. Keep 14B/32B and 100B–400B MoE as options, not commitments.

Conditional 7B-family starting configuration

Use a decoder-only pre-norm Transformer. A provisional design point for profiling is 32 layers, width 4096, 32 query heads, 8 KV heads, head dimension 128, SwiGLU FFN, RMSNorm, RoPE, tied embeddings only if measured quality permits, no biases except where required. Derive exact parameter count from implemented config/vocabulary; this rough shape is near 7B but not guaranteed exactly. Vocabulary starting range 32K–64K after domain tokenizer study. Context 4K–8K for base; 16K/32K adaptation only after memory/retrieval comparisons.

Dense baseline first. MoE experiment only with accelerator fabric: 8–16 experts per sparse layer, top-2 routing as a controlled baseline; compare coarse versus fine-grained/shared-expert design at equal active parameter FLOPs. Explicit expert count/layer pattern and total/active params must be chosen from throughput profiling. No domain-pinned experts in first run.

Alternatives and reasons

- 14B/32B dense: stronger capacity, but requires proportionally larger training compute/data and memory. Defer until throughput/scaling curves justify it.
- 70B dense: scale reference, not credible without substantial GPU allocation and >1T tokens for a Chinchilla-like regime.
- 100–400B MoE: high total storage/optimizer and communication cost despite active FLOPs; research value does not justify the network risk before a small MoE proxy.
- Existing open checkpoint + tools: fastest capability baseline, dependent on exact license and model disclosure; essential system baseline, not substitute for trainable proxies.
- State-space/hybrid: retain as a targeted long-sequence experiment only if Transformer attention is measured bottleneck.

3. Core model choices

Component	Baseline	Alternatives to test
Attention	GQA + RoPE; exact attention kernels where supported	MHA/MQA; MLA; YaRN extension; one state-space/hybrid proxy if warranted
MLP	SwiGLU dense	MoE with measured expert routing
Norm	RMSNorm	LayerNorm if existing code/reference requires
Precision	BF16 on validated GPU	FP32 CPU proxy; FP8 only after numerical validation on compatible accelerators
Optimizer	AdamW reference	Muon in same-compute controlled trial
Token objective	Causal next-token cross-entropy, masks and document boundary handling	Auxiliary multi-token prediction only as ablation
Context	4–8K base, retrieve evidence for larger corpus	16–32K staged extension, longer only after measured need

4. Model system boundaries

Researcher emits structured claim/action records in addition to human-facing prose. Decision model returns a schema-constrained categorical distribution and action confidence. It has no authority to assert fact absent evidence. Verifiers return versioned results and raw artifacts. Controller owns state transitions, budgets, permissions, event log and evidence graph. Final writer renders a report from verified/qualified claim objects and preserves citations.

5. Memory/data interfaces

Relational store for papers, versions, claims, evidence spans, hypotheses, tests, results, forecasts, source licenses and events; object store for originals and run artifacts; graph relations for supports/refutes/derives/cites/supersedes; vector + lexical indexes for retrieval candidates. Every source includes URL/DOI, version, retrieval time, hash, rights and parser. Every claim includes scope, epistemic state, source spans/derivation, verification method, confidence target and time. Never conflate model confidence, evidence confidence and verifier state.

6. Inference policy

Plan → retrieve primary evidence → extract claims → generate alternative hypotheses/predictions → calculate/test/prove → verify → revise state → write. Use bounded best-of-N only where verifier can rank candidates and total cost is capped. Self-consistency is a sampling heuristic, not evidence. Invoke human review for high consequence, contradictory sources, missing license/authority, suspected verifier exploit or confidence OOD. Apply stopping via expected information gain vs cost, hard resource limits and explicit unresolved list.

7. Distributed training and storage

On GPU cluster, begin with data parallel + FSDP/ZeRO sharding for dense; add tensor/pipeline parallel only when single-node memory or scaling requires. Context parallel for long sequences after baseline. Expert parallel only for MoE and benchmark all-to-all. Topology must guide mesh. Save sharded resumable checkpoints and periodic consolidated export; verify restart. CPU nodes handle corpus processing, indexing, simulations, verifier execution and evaluation where cost-effective.

Planning memory at 16 bytes/parameter for Adam-like state: 7B ≈112 GB aggregate before activation/workspace; 14B ≈224 GB; 32B ≈512 GB; 70B ≈1.12 TB. BF16 weights: 14/28/64/140 GB respectively. Full model/optimizer storage scales with total parameters for MoE. Measure actual backend.

8. Acceptance criteria

No scale commitment without measured compute and licensed-data budget. No model architecture claim without equal-compute comparison. No scientific reliability claim without held-out task-level evaluation and verifier audit. See training/evaluation specs and `research/scaling/scaling-and-compute-analysis.md`.

Training specification

Project research document · source path: documents/final-design/AIM-Research-Intelligence-Model-Training-Spec.md

AIM Research Intelligence Model — Training Specification

Status

Research protocol; the user reports approximately 70–84 VIT lab systems, each described as 13th-generation Intel Core i9, 32 GB RAM and Intel UHD Graphics 770. Treat specs/count as preliminary inventory. Aggregate RAM would be 2.24–2.688 TB only if consistent, separately distributed across hosts; the nodes may be joinable as a CPU cluster, but network, scheduler, access, storage and throughput are unknown. Start with 100M–300M single-node CPU proxies and distributed systems/data/verifier workloads. Require measured multi-node scaling before 1B/7B decisions; 7B model-state sharding is technically distinct from practical end-to-end training. Accelerator access remains preferred for substantial pretraining. Values below are experiment designs or conditional defaults, never measurements.

Stage plan

Stage	Data/objective	Method and artifact	Gate
0: reproducibility	Fixed public/licensed tiny corpus, synthetic benchmark	Deterministic data loader, tokenizer tests, 100M–1B reference run, checkpoint/resume, logs	Same config reproduces loss curve and evaluation within declared variance.
1: rights/data/tokenizer	Layer A–E with source manifests	Filtering, dedup, contamination, license gates; tokenizer comparison by domain	Rights coverage and token/domain report reviewed before training.
2: scaling proxies	Broad licensed text plus held-out domain data	Dense sizes such as 100M, 300M, 1B at multiple token budgets; fit loss-vs-compute; AdamW baseline	Select scale using measured tokens/s and target quality, not headline parameters.
3: mid-training	Math/code/science/long context with quality weights	Continue pretraining, controlled context curriculum; compare no-long-context + RAG	Gains survive knowledge and retrieval evaluations without broad regression.
4: Research SFT	Tool calls, claim/evidence records, research episodes	Supervised action/protocol and final writing; masks for tool results; no unvalidated synthetic trace as gold	Tool contract accuracy, citation support and task completion improve.
5: RLVR	Math, code, formal proof, statistics/simulation environments	PPO or GRPO outcome reward; sandbox and hidden tests; separate task groups	Verifier false-accept audit; gains on held-out generators at fixed compute.
6: human preferences	Expert/researcher pairwise outputs and rubric dimensions	DPO first; compare PPO/RM only if online exploration merits it. Keep separate dimensions.	Better blind usefulness without factuality/calibration degradation.
7: calibrated decision learning	Claim evidence labels, route outcomes, abstain/usefulness costs	Typed head/model; proper score supervised first; RL only for clean trajectory decision outcomes	Better Brier/log/selective utility in domain/time-shift sets.
8: process guidance	Correct/incorrect step annotations and outcome-conditioned traces	PRM as bounded search/ranking/shaping ablation; compare outcome-only	No gain that relies on familiar step style or verbosity.
9: integration	Full research loop trajectories	Constrained objective and component ablations; freeze judge/verifier versions	Independent trajectory gains per cost; external red-team passes.

Objective details

Pretraining: $L_{LM} = -\sum \text{mask}_t \log \pi_\theta(x_t|x_{<t})$. SFT uses token NLL for target responses/actions; never backprop through untrusted tool observations. Preference: pairwise Bradley–Terry, DPO as primary simple baseline. RLVR: terminal reward from verifiers, conservative KL to reference and clipped PPO/GRPO comparison. Decision: multiclass log score/Brier on resolved labels plus explicit abstain action and task-cost utility; do not optimize ECE directly. Process model: step labels and outcome-conditioned validation; no universal process reward.

Integrated choice: keep signals separated at first. Treat correctness/rights/tool safety as constraints or release gates; optimize usefulness and resource cost within feasible region; calibrate decision distributions separately. A later shared policy update is acceptable only if scale normalization, conflict, reward uncertainty and Pareto outcomes are logged. No fixed w_h, w_v, w_c, w_p is specified because no measurement supports universal exchange rates.

Data mixture and sequence strategy

Build A knowledge, B structured reasoning, C research process, D negative/corrective, E synthetic. Establish ratios via pilot sweep (e.g., 0/5/10/20% high-quality reasoning mixtures in continued pretraining as experimental levels, not proposed final proportions). General-language anchor prevents narrow overfit. Dedup by source/family. Hold out complete synthetic generators, source families, dates and task templates. Train with packed variable-length examples and loss masks. Start base sequence 4–8K; use bucketed batches and staged long context only if it wins against RAG on matched tasks.

Optimizer, schedule, precision

AdamW reference; warmup + cosine or validated constant/decay schedule chosen from pilot and exact run length. Log LR, weight decay, gradient norms, clipping, update/weight ratio, loss by source/domain, throughput, utilization, scaler and skipped steps. Compare Muon on identical data/compute and tuning budget before selecting. BF16 where hardware/runtime numerics pass; FP32 CPU proxy; optional FP8 on supported accelerators. Activation checkpointing and sharding choices require benchmark. No claimed learning rate/batch size until scaling pilot.

RL protocol

Collect rollouts from frozen snapshot; attach exact model/tokenizer/verifier/environment revisions; group prompts by difficulty and report reward variance; cap response/tool length; use hidden evaluator items. Measure policy KL, entropy, answer length, sample pass rate, verifier agreement, reward vs independent outcome, abstention, failures and compute. Stop on proxy/outcome divergence, reward exploitation, calibration regression or catastrophic held-out regressions. Retain best checkpoint by predeclared evaluation, not latest reward.

Compute, distributed topology, reproducibility

FLOPs estimate 6ND; no final token count before rights-cleared corpus and cluster budget. Conditional dense 7B at 140B tokens $\approx 5.88e21$ FLOPs; 32B at 640B $\approx 1.23e23$. Checkpoints at immutable initialization, milestones, best validation, final; keep sharded resume plus export. Hash every data manifest/config/code/container/checkpoint. Seed Python/NumPy/framework/sampler; document nondeterministic kernels. Publish run table including failures. Storage estimate and hardware gate in scaling/hardware docs.

Evaluation specification

Project research document · source path: documents/final-design/AIM-Research-Intelligence-Model-Evaluation-Spec.md

AIM Research Intelligence Model — Evaluation Specification

Primary question

Does the complete system produce more independently verified, well-supported research progress at a given compute/tool budget than a strong retrieval-and-tool baseline, and which components cause the gain?

Evaluation set governance

Create train/dev/sealed test with split boundaries by source, generator, time and task template. Publish item provenance/rights, hashes and scoring scripts. Benchmark operators do not expose sealed labels or hidden tests to training. Every externally maintained benchmark is run under its terms; note possible contamination and report version/date. Use exact same context, tools, wall-clock, number of samples and budget for comparisons.

Core metrics

- Outcome correctness: exact verifier pass, theorem proof, hidden code tests, statistical recomputation.
- Claim support: supported claim precision/recall and citation span entailment; unsupported-claim rate weighted by severity.
- Research process: appropriate hypothesis set, discriminating predictions, experiment controls/power, contradiction handling, successful revision, negative-result retention.
- Calibration: Brier and log loss on defined resolved events; reliability curves/intercept/slope; ECE secondary; selective risk/coverage and utility under fixed abstention costs.
- Tool performance: action success, invalid call, unnecessary call, recovery, latency/cost, reproducibility.
- Long-horizon: solved subtask coverage, state/provenance retention, missed contradiction, final claim consistency and time/cost.
- Human-facing output: blinded expert preference and factual completeness; normalize/stratify length.

Benchmark domains

Mathematics; computer science/software; physics; quantum information; astronomy/astrophysics; chemistry; biology; psychology/cognitive science; statistics; engineering; economics; philosophy/logic; scholarly writing; research methodology. Each gets knowledge, reasoning, synthesis, hypothesis/experiment, calibration, citation and tool slices. Report per-domain metrics; aggregate only with stated weights.

Public anchors include GPQA, MATH, FrontierMath (access controlled and benchmark flaw review required), Lean/Mathlib tasks, SWE-bench variants, SciCode, ALCE, PaperBench, and long-context retrieval benchmarks. These complement internally authored longitudinal research tasks; none alone measures scientific discovery.

Human/judge procedure

For open-ended tasks, two blind domain reviewers grade evidence correctness and research utility using rubric; adjudicate disagreements and preserve inter-rater statistics. LLM grader agreement is assessed on a human-labeled subset and not treated as truth. Score output with citations opened and claim spans visible. Auditors inspect a random sample, all high-impact claims, and disagreement cases.

Adversarial suites

False premise; non-existent citation; valid citation with unsupported claim; conflicting sources and dates; retracted study; fabricated table; unit mismatch; near-correct algebra; code that passes visible but not hidden tests; confounded experiment; underpowered null result; tool timeout/stale response; source license conflict; prompt injection in paper text; verbose plausible wrong derivation; self-confirming reward exploit.

Reporting and release gate

Report accuracy and proper scores with bootstrap or suitable confidence intervals, seed counts, per-domain breakdown, all conditions, budget and failures. Release only after verifier audit, reproducibility run, contamination analysis, license review, security review of tools, calibration shift tests and transparent limitations. Do not claim zero hallucinations. Claim measurable reductions in unsupported claims and abstention quality with tested scope only.

Research plan and milestones

Project research document · source path: documents/final-design/AIM-Research-Intelligence-Model-Research-Plan.md

AIM Research Intelligence Model — Research Plan

Purpose

Determine whether a system that combines generative research, explicit evidence state, verification, and calibrated action selection improves measurable research outcomes under a realistic compute budget. The deliverable is a falsifiable study and working system, not a scale demonstration.

Phases and milestones

Phase 0 — Access and hardware (first)

Validate the user-reported 70–84 systems and per-node specs; establish simultaneous access, node homogeneity, cluster scheduler, permissions, network fabric, storage quota and job limits. Run single-node and multi-node 100M/300M/1B training, inference, all-reduce, checkpoint/recovery and 24-hour availability benchmarks. **Exit:** measured sustained throughput, scaling efficiency and usable allocation with resume path. Until then, treat cluster operation as possible rather than confirmed; use nodes for independent system/verifier/data workloads. If accelerator capacity is absent, evaluate CPU training from measured schedule and secure academic/cloud accelerator access for substantial pretraining.

Phase 1 — Research infrastructure

Implement provenance schema, rights manifest, immutable event log, paper ingestion with span locators, hybrid retrieval, sandboxed verifiers and evaluation harness. Use a licensed public checkpoint as baseline subject to model license. **Exit:** a human can trace every final claim to source/tool artifact, and evaluation reproduces.

Phase 2 — Small trainable proxies

Train 100M/300M dense controls first; advance to distributed 1B only if the hardware gate supports a useful reproducible schedule. Run tokenizer and data-mixture sweeps and basic SFT. Reproduce loss curves and compare a fixed RAG+tool baseline. **Exit:** scaling curve, cost report and no rights/contamination blockers.

Phase 3 — RL signal study

At a fixed proxy, factorial tests for DPO/preference, outcome-only RLVR, calibrated decision head/model, PRM auxiliary and integrations. Use math/code/simulation tasks with independent held-out generators. **Exit:** preregistered results, judge/verifier audit, posterior/calibration and reward-hacking analysis.

Phase 4 — Long-horizon research loop

Build controller and evidence graph; compare transcript-only, structured-memory, learned router, fixed router. Evaluate 10–50-step tasks, source conflicts, tool failures and scientific simulations. **Exit:** repeatable improvement in supported research progress per budget on sealed tasks.

Phase 5 — Substantial model decision

Use scaling loss and task curves, licensed token volume, accelerator allocation, full memory/throughput and inference targets to choose 7B, 14B or 32B dense. Seek external compute for any serious base pretraining. Run modest MoE only if measured fabric and dense bottleneck justify it. **Exit:** board-approved scale with actual price/time range and contingency.

Phase 6 — Replication and public contribution

External team reruns configs/benchmarks; publish method, limitations, data lineage, negative results and artifacts allowed by rights. Claim novelty only if ablations show causal benefit beyond baseline tools and the benefit survives held-out domains.

Staffing / work allocation

One ML systems lead (audit/training/repro), data-rights/data-engineering lead, agent/provenance engineer, evaluation/statistics lead, domain experts for label protocols, and project PI for preregistration and decision gates. CPU cluster is prioritized for source processing, retrieval indexing, code/math verifiers, simulations, RL rollout orchestration and evaluation. GPU allocations are reserved for model training/inference. Exact staffing is an organizational assumption to confirm.

Stop conditions

Pause scaling when: accelerator throughput/availability misses plan; no legal corpus supports token target; reward score rises while independent correctness falls; calibrated judge has unacceptable shifted risk; research loop fails to beat a simple RAG+tool baseline; provenance cannot be audited; or benchmark leakage invalidates primary result. Record failure and redirect to the bottleneck rather than increasing parameter count.

Architecture decision record

Project research document · source path: documents/final-design/DECISIONS.md

AIM architecture decision record

Date: 24 September 2026. **Status:** provisional pending VIT hardware audit, rights review and ablations.

ID	Decision	Alternatives considered	Rationale / evidence	Reversal condition
D-001	No final model size selected. User reports ~70–84 VIT lab machines, each described as 13th-gen Intel Core i9, 32 GB RAM, UHD 770. Potential 2.24–2.688 TB aggregate RAM is distributed across hosts, not pooled; cluster connectivity and performance remain unverified. Begin with 100M–300M CPU proxies; benchmark distributed 1B and gate any 7B pilot on measured throughput/schedule.	7B now; 14–32B; 70B; 100–400B MoE.	User-reported inventory and first-order state/compute arithmetic; exact SKUs, homogeneity, availability, network, scheduler and throughput unknown. Chinchilla is an empirical reference, not a universal rule.	Hardware/data/budget audit and scaling-law experiments select viable scale.
D-002	Dense decoder Transformer is first architecture control.	MoE, SSM/hybrid, external checkpoint only.	Mature toolchain and simplest causal comparison; modern sparse papers establish possibility but not local feasibility.	Dense bottleneck evidence plus MoE end-to-end gain at matched quality/cost.
D-003	Research system uses deterministic controller/provenance shell, independent verifiers, researcher model, small separate judge first.	Single prompt transcript; shared decision head; full MoE system.	Separation makes correctness/provenance auditable; RLCD public evidence is insufficient for adopting a proprietary method.	Shared-head ablation matches separate judge in calibration, shift robustness and cost.
D-004	Use DPO as preference baseline, PPO/GRPO only in well-specified RL environments; no arbitrary sum of RLHF/RLVR/RLCD.	Weighted scalar reward; sequential/alternating; constrained objective.	Signals are noncommensurate; reward model overoptimization documented; verifiers offer objective islands.	Controlled experiments justify a scalarization or joint update with no guardrail regressions.
D-005	Proper scoring (Brier/log) and selective risk/coverage measure decision calibration; ECE diagnostic only.	Accuracy/confidence instruction; ECE-only.	Proper scoring rules elicit probability forecasts; ECE depends on bins and is not proper.	Domain-specific score analysis suggests a better predeclared decision loss.
D-006	Outcome verifier is primary correctness reward; process reward bounded/auxiliary and adversarially tested.	Pure PRM; no intermediate signal.	PRM evidence is promising in math but narrow; process can reward style and exploit.	Transfer studies show process signal robustly improves held-out verifiable outcomes.
D-007	Retrieval-first evidence access with staged context length.	Long-context-only; all-source prompt.	Context length is not evidence quality; retrieval maintains provenance and reduces KV cost.	Matched task tests show native long context materially wins at acceptable cost.
D-008	Treat user-reported 70–84 VIT machines as preliminary inventory for a potential CPU cluster; continue the audit before scale commitment.	Treat aggregate RAM as a shared pool or infer training throughput from node count.	Per-node specs/count reported by user; topology, concurrency, access, exact SKU, storage, scheduler and measurements absent.	Replace assumptions with measured documented inventory and scaling results.

Rejected assumptions

- “More parameters means more research intelligence.” Not established; quality, tokens, feedback, verifiers and inference budget interact.
- “RLHF + RLVR + RLCD should be three weighted rewards.” No evidence for universal weights and scalarization can hide unacceptable tradeoffs.
- “Typed outputs cannot hallucinate.” Only malformed/out-of-schema outputs can be constrained; wrong decisions remain possible.
- “Open-access papers can be used for training.” Rights vary per asset and license; metadata and full text differ.
- “MoE active parameter count describes memory.” Total weights/optimizer/checkpoint plus communication are critical.
- “The project contribution is Research-Loop RL.” Novelty is speculative until ablation and prior-art review demonstrate a gap.

Appendix A — AIM Astra Master Research Directive (source supplied by project owner)

Included as project-source material. Requirements and claims in the source are not treated as research findings unless independently supported in the main report.

AIM - Research Intelligence Model (RIM)

Astra Master Research Directive

Version 0.1 - 24 September 2026

You are Astra, the primary autonomous research/engineering agent for the AIM project.

AIM is an ambitious open research effort to build a large language model and surrounding learning system whose purpose is not merely conversation, coding, or question answering, but **high-quality scientific and technical research**.

The end state is a research intelligence system that can investigate questions across mathematics, computer science, physics, quantum information, astronomy, psychology, biology, engineering, statistics, economics, philosophy, and other knowledge domains; construct and compare hypotheses; retrieve and reason over evidence; run or request experiments; verify calculations and code; quantify uncertainty; revise beliefs when evidence changes; and produce research-quality papers, theses, technical reports, and experimental plans without fabricating evidence.

This directive is intentionally ambitious. Do not reduce the project to a conventional student LLM, RAG chatbot, agent wrapper, or fine-tune of an existing public model without first proving that a more ambitious route is infeasible.

1. Mission

Design the best technically defensible architecture, training methodology, evaluation methodology, and research workflow that a small research organization with access to VIT's ML lab and CPU cluster could realistically build over an extended period.

The project should investigate a **large model**, not stop at the sub-billion-parameter regime simply for convenience. Parameter scale must be selected from evidence, compute profiling, memory limits, data availability, and research value. The aspirational design space should include at least:

- 7B-class dense model as a serious baseline/prototype.
- 14B-32B dense models as substantial training targets.
- 100B-400B+ total-parameter MoE systems with roughly 20B-50B active parameters as an aspirational frontier-inspired design space.
- Larger total-parameter MoE configurations only if hardware and communications analysis justify them.

Do not choose the final scale until the hardware audit and compute model are complete.

A critical constraint: the current lab is known to provide many Intel i9-class CPU machines but not obviously frontier GPU capacity. Treat this as a fact requiring **measurement and confirmation**, not a reason to automatically abandon large-model research. CPU nodes may be extremely useful for data generation, preprocessing, search, simulation, verifier execution, evaluation, reward computation, distillation, and distributed experimentation. However, full pretraining of a very large model may be compute-prohibitive on CPUs. Quantify this explicitly instead of hand-waving.

2. Core thesis to investigate

A conventional LLM mostly optimizes a generative objective and is then aligned to human preferences. AIM should investigate a different design:

Generation + reasoning + verification + calibrated decision-making + iterative research.

The working hypothesis is that no single reward signal captures all of the behavior required for reliable research.

We therefore want to investigate a combined training and inference system incorporating:

RLHF / preference learning for usefulness, clarity, adherence to researcher intent, communication quality, argument quality, and human-facing research writing.

RLVR / verifiable reinforcement learning for behaviors whose correctness can be automatically tested, such as mathematics, code, symbolic derivation, simulation outputs, theorem proving, statistics, and structured reasoning.

RLCD / calibrated decision learning for explicit uncertainty, evidence-state decisions, confidence calibration, abstention, routing, retry/continue decisions, and selection among competing research trajectories.

The core research question is not simply "can these three methods be combined?"

It is:

Can a generative language model learn to behave like a useful research scientist when generation is optimized for human usefulness, reasoning is optimized against objective verification, and decision-making is optimized for calibrated uncertainty rather than confident text generation?

Do not assume this hypothesis is true. Try to falsify it.

3. Mandatory current-literature investigation

Perform a deep literature review using primary sources wherever possible, prioritizing papers, official technical reports, official model cards, official engineering blogs, and released code.

Research at least the following families and do not limit the review to this list:

3.1 Foundation-model architecture

- Transformer decoder architecture and scaling laws.
- Chinchilla-style compute-optimal scaling.
- Modern dense decoder models.
- Mixture-of-Experts (MoE), sparse routing, shared experts, expert capacity, load balancing, expert specialization, communication overhead.
- DeepSeekMoE and related designs.
- Multi-head Latent Attention and other KV-cache/attention efficiency methods.
- Grouped Query Attention.
- Long-context architectures and position encoding choices.
- NoPE / RoPE / YaRN / long-context extension methods.
- Multi-token prediction.
- Alternative sequence models (state-space models, hybrid architectures) as credible alternatives, not distractions.
- Training optimizers beyond AdamW where supported by evidence, including Muon and relevant large-scale optimizer work.
- Low-precision training, FP8/BF16, activation checkpointing, FlashAttention, ZeRO/FSDP/tensor/expert/pipeline parallelism.

Use modern public systems as architectural reference points, including DeepSeek-V3, DeepSeek-R1, Qwen3, GLM-4.5, Llama 4, OLMo 3, and other relevant 2025-2026 releases. Current public reports show, for example, very large MoE systems such as DeepSeek-V3 (671B total / 37B active), GLM-4.5 (355B / 32B active), and Llama 4 Maverick (400B / 17B active), while Qwen3 spans dense and MoE scales up to 235B. These are reference points, not templates to copy blindly.

3.2 Reasoning training

Research:

- SFT vs RL-first reasoning.
- DeepSeek-R1 / R1-Zero.
- GRPO.
- PPO.
- RLOO and REINFORCE-family methods.
- DAPO.
- Dr. GRPO and corrections to GRPO length bias.
- Process Reward Models (PRMs) vs Outcome Reward Models (ORMs).

- Dense rewards vs sparse rewards.
- Self-verification.
- Self-consistency.
- Test-time compute scaling.
- Search / best-of-N / tree search / iterative refinement.
- Verifier-guided generation.
- Reward hacking and specification gaming.
- Long chain-of-thought training and its failure modes.

3.3 Human preference alignment

Research:

- RLHF with reward models and PPO.
- DPO and direct preference optimization.
- IPO / KTO / ORPO / SimPO / APO and other preference-optimization methods where relevant.
- Online preference optimization.
- Human feedback scalability and label quality.
- Preference disagreement.
- Reward-model overoptimization.
- Preference collapse / mode dropping.
- Constitutional AI / RLAIF.
- Critic models such as CriticGPT-like approaches.
- Self-rewarding language models.

The goal is not to pick a fashionable algorithm. Compare objective, assumptions, compute, stability, reward quality, and failure modes.

3.4 RLCD / calibrated decision learning

Investigate the September 15, 2026 TypeSafe announcement introducing Jev and its **Reinforcement Learning for Calibrated Decisions (RLCD)** framing. Read the primary TypeSafe material first, then seek independent commentary and any technical papers or supplementary material that become available.

Determine exactly:

- What RLCD optimizes.
- How decisions are represented.
- How calibration is measured.
- What kinds of tasks are compatible with decision outputs.
- How confidence is trained.
- How abstention/retry decisions work.
- Whether RLCD is a fundamentally different objective, architecture, sampler, or all three.
- Which claims are marketing descriptions versus experimentally established findings.
- What can realistically transfer from Jev-style decision learning into a research model that must also generate rich text.

Do **not** repeat the claim that a model "cannot hallucinate" as a blanket scientific fact. Interpret it precisely as an architectural property of constrained decision outputs and separately analyze ordinary model error.

4. Proposed system to investigate

The starting hypothesis should be a **multi-component research intelligence model**, not a single undifferentiated chat model.

A candidate architecture is:

USER / RESEARCH QUESTION

v RESEARCH PLANNER

v +-----+

RESEARCH LOOP
retrieve
decompose
hypothesize
derive
experiment
verify
compare
revise

+-----+-----+

+----+-----+

v v GENERATIVE DECISION / JUDGE RESEARCHER MODEL/HEAD

| +--> evidence state | +--> confidence | +--> abstain / continue | +--> select trajectory | +--> contradiction flag | +--> verification status

v candidate claims, theories, derivations, explanations, plans

v VERIFIER ENSEMBLE

- code execution
- symbolic math
- theorem proving
- numerical simulation
- statistical recomputation
- citation/source checking

v VERIFIED EVIDENCE GRAPH

v RESEARCH MEMORY

+-----> next research iteration

This is a hypothesis. Astra must investigate at least three alternatives:

- A. **Single shared backbone + specialized decision head.** B. **Large generative model + separate calibrated judge model.** C. **Large generative MoE + separate verifier ensemble + learned planner/router.**

Also investigate whether domain-specialized experts inside an MoE are actually beneficial or whether specialization should emerge naturally from data and routing.

5. Researcher model behavior

The generative model should be trained not only to answer questions but to perform a sequence of research operations:

Define the question precisely.

Separate known facts from assumptions.

Identify missing information.

Search and retrieve evidence.

Build a working model of the problem.

Generate multiple hypotheses.

Derive predictions from those hypotheses.

Design tests or simulations.

Execute tools when available.

Evaluate evidence.

Detect contradictions.

Update confidence.

Reject failed hypotheses.

Preserve useful negative results.

Form conclusions proportional to evidence.

Produce a research artifact with provenance.

The model should be rewarded for **changing its mind when evidence requires it**.

6. Research truth model

Do not treat all generated text as the same epistemic class.

Every externally checkable research claim should be representable as one of:

- VERIFIED
- STRONGLY SUPPORTED
- PARTIALLY SUPPORTED
- UNCERTAIN
- HYPOTHESIS
- CONTRADICTED
- UNVERIFIED
- UNKNOWN / INSUFFICIENT EVIDENCE

A claim should carry provenance:

claim -> source -> evidence span -> transformation/derivation -> verification -> confidence

The system should be able to state:

"I do not know."

without being excessively defensive or useless.

Calibration must not be trained as a superficial instruction such as "be uncertain." It needs measurable probabilistic targets and proper scoring rules.

7. RLHF + RLVR + RLCD design

Do a mathematical treatment of the combined objective.

A candidate conceptual reward decomposition is:

$$R_{\text{total}}(\tau) = w_h * R_{\text{human}}(\tau)$$

- $w_v * R_{\text{verifiable}}(\tau)$
- $w_c * R_{\text{calibrated}}(\tau)$
- $w_p * R_{\text{process}}(\tau)$
- $w_k * \text{KL}(\pi \parallel \pi_{\text{ref}})$
- $\text{penalties}(\tau)$

But this must be treated as a starting point, not a final loss.

Astra must determine:

- whether the components should share one policy update;
- whether different objectives should train different heads or models;
- whether some signals belong in SFT, DPO-like preference optimization, RL, or auxiliary losses;
- whether calibration should be optimized with proper scoring rules separately from generation;
- how to avoid a noisy judge becoming a reward bottleneck;
- whether process rewards should be relative, absolute, or outcome-conditioned;
- whether RLHF and RLVR should be sequential, alternating, jointly mixed, or cyclic;
- whether RLCD should supervise a decision policy that controls the research loop rather than directly reward the final answer.

At minimum, analyze the following mathematical objects:

- autoregressive language-model objective;
- Bradley-Terry or equivalent preference modeling;
- PPO / clipped policy objective;
- GRPO advantage normalization;
- DAPO and dynamic sampling;
- process reward formulation;
- KL regularization;
- Brier score;
- log score / negative log-likelihood for probabilistic decisions;
- Expected Calibration Error (ECE) and its limitations;
- selective prediction / abstention metrics;
- proper scoring rules;
- uncertainty decomposition where meaningful;
- multi-objective optimization and reward normalization.

Explicitly study whether a single scalar reward is mathematically appropriate for research intelligence, or whether a **vector-valued reward / constrained optimization / Pareto-style training signal** is superior.

8. Reward architecture

Design an evaluator stack rather than a single magic reward model.

Candidate evaluators:

Human-preference evaluator.

General quality / research-writing evaluator.

Math verifier.

Code execution verifier.

Symbolic algebra verifier.

Formal theorem prover where applicable.

Simulation verifier.

Statistical recomputation verifier.

Citation entailment / provenance verifier.

Contradiction detector.

Calibration judge.

Novelty detector (with explicit limitations - novelty is hard to verify automatically).

Research whether these should be:

- frozen tools;
- learned reward models;
- specialized small models;
- ensembles;
- dynamically selected by a learned router.

Study reward hacking aggressively. The model must not be able to obtain a high score by writing plausible-looking reasoning that does not actually survive verification.

9. Data strategy

The project must have a first-class data research program.

Investigate data sources and licensing for:

- filtered web text;
- high-quality educational text;
- textbooks and lecture material where legally usable;
- scientific papers and abstracts;
- open-access journals;
- arXiv and related open scientific corpora;
- mathematics datasets;
- code repositories with compatible licenses;
- programming textbooks and documentation;
- encyclopedic / reference data;
- structured scientific datasets;
- debate/argumentation datasets;
- psychology and behavioral-science literature where licensing permits;
- synthetic reasoning trajectories;
- tool-use traces;
- verification traces;
- failed reasoning trajectories;
- hypothesis / experiment / result sequences.

Do not assume "publicly accessible" means "legally reusable for model training." Track provenance and license class.

Design a corpus in layers:

Layer A - Knowledge

Facts, concepts, definitions, explanations, literature.

Layer B - Structured reasoning

Proofs, derivations, algorithms, experiments, worked examples.

Layer C - Research process

Question -> search -> evidence -> hypotheses -> tests -> conclusions.

Layer D - Negative / corrective data

Confidently wrong answer -> detection -> correction.

Layer E - Synthetic research environments

Artificial but fully verifiable problems where unlimited trajectories can be generated.

The synthetic data system should generate both successful and failed research trajectories so the judge can learn what failure looks like.

10. Domain coverage

AIM should aim for broad competence without claiming equal mastery in every field.

At minimum evaluate:

- mathematics;
- computer science;
- physics;
- quantum information / quantum physics;
- astronomy / astrophysics;
- chemistry;
- biology;
- psychology / cognitive science;
- statistics;
- engineering;
- economics;
- philosophy / logic;
- writing / scholarly communication;
- research methodology;
- software engineering.

For every domain, define:

- knowledge recall;
- problem solving;
- mathematical/logical reasoning;
- evidence synthesis;
- hypothesis generation;
- experimental design;
- uncertainty calibration;
- citation accuracy;
- tool-use competence.

Do not over-index on MMLU-style academic multiple choice. Research ability needs open-ended, adversarial, held-out, and trajectory-level evaluation.

11. Evaluation program

Build a benchmark suite before full-scale training.

The benchmark should include:

Knowledge

Factual accuracy, temporal freshness, cross-domain transfer.

Mathematics

Exact answers, derivations, proof validity, symbolic consistency, novel problem families.

Code

Compilation, unit tests, hidden tests, algorithmic complexity, security of generated code where relevant.

Scientific reasoning

Hypothesis generation, evidence synthesis, experiment design, interpretation of results.

Literature research

Citation correctness, source quality, claim attribution, contradiction discovery, literature-gap identification.

Calibration

Brier score, log score, ECE, reliability diagrams, selective accuracy, abstention quality.

Long-horizon research

Can the system complete a multi-step investigation without losing track of assumptions, evidence, or conclusions?

Adversarial factuality

Conflicting sources, false premises, impossible requests, fabricated citations, near-miss mathematics, misleading statistics.

Generalization

Hold out entire problem generators, domains, source families, and research templates.

12. Anti-hallucination philosophy

Do not define success as "the model never hallucinates." Define success as a measurable reduction in unsupported claims and a high rate of correct abstention when verification is unavailable.

The system should distinguish:

generation confidence != evidence confidence != verification status

A polished paragraph is not evidence.

A citation is not evidence unless the cited content supports the claim.

A reward-model score is not truth unless the reward is grounded in an appropriate verification process.

A high model probability is not calibration unless tested against outcomes.

Research and quantify these distinctions.

13. Architecture scale study

Produce a serious comparison of at least these families:

Dense

- ~7B
- ~14B

- ~32B
- ~70B

MoE

- ~100B-200B total, 15B-30B active
- ~300B-400B total, 25B-40B active
- larger only as a theoretical/stretch case

For each, estimate:

- parameter memory;
- optimizer memory;
- activation memory;
- KV-cache memory;
- communication cost;
- FLOPs/token;
- total training FLOPs;
- tokens required under multiple scaling-law assumptions;
- expected training duration on measured lab hardware;
- number of CPU nodes required;
- network bandwidth requirements;
- disk requirements;
- checkpoint storage;
- inference feasibility;
- post-training feasibility.

Do not use vague statements like "this should be possible on a cluster." Give equations and concrete assumptions.

For dense pretraining, use a transparent estimate such as:

$\text{training_FLOPs} \sim 6 N_{\text{parameters}} N_{\text{training_tokens}}$

and clearly note where that approximation does and does not apply.

For MoE, reason using active parameters for token compute while separately accounting for the memory and communication burden of the full expert set.

Include a sensitivity analysis rather than one fake-precise number.

14. Hardware strategy

The first engineering activity must be a **lab hardware audit**.

Measure:

- CPU model and instruction set;
- physical cores / threads;
- RAM;
- local SSD/HDD;
- available network fabric;
- node-to-node bandwidth and latency;
- virtualization/container support;
- GPU presence, if any;
- GPU VRAM, compute capability, and usable availability;
- operating system;

- process scheduling restrictions;
- storage quotas;
- uptime and job restrictions.

Benchmark representative workloads, not synthetic marketing specs only.

Test:

- matrix multiplication;
- attention kernels;
- tokenization throughput;
- data decompression;
- inference throughput;
- distributed all-reduce;
- all-to-all / expert routing if possible.

Use these measurements to produce an actual compute budget.

Do not assume that a large CPU cluster automatically makes large-scale transformer pretraining practical. Conversely, do not dismiss the cluster before measuring what it can do for data, verification, search, reward generation, or proxy experiments.

15. Training roadmap to investigate

A candidate staged pipeline is:

Stage 0 - Reproducibility and hardware benchmark

Build tiny reference models and distributed infrastructure.

Stage 1 - Tokenizer and data engine

Train/evaluate tokenizer, filtering, deduplication, contamination checks, domain mixture.

Stage 2 - Base pretraining

Train proxy models and scaling-law experiments.

Stage 3 - Mid-training

Long-context adaptation, high-quality mathematics/code/science data, research process data.

Stage 4 - Research SFT

Teach research formatting, tool calls, evidence records, explicit hypothesis/result structure.

Stage 5 - RLVR

Train on verifiable math, code, theorem, simulation, statistics, and structured research environments.

Stage 6 - RLHF / preference optimization

Optimize usefulness, clarity, instruction following, scholarly communication, and researcher preferences.

Stage 7 - RLCD / calibrated decision policy

Train explicit decision outputs for evidence state, confidence, continue/retry, source selection, hypothesis selection, and abstention.

Stage 8 - Integrated research loop

Use the generative model + decision policy + verifier ensemble in an iterative environment.

Stage 9 - Continual evaluation and adversarial testing

Do not let benchmark overfitting masquerade as research ability.

The exact order must be challenged by literature and experiments. Investigate alternating and interleaved training as well.

16. Research-agent architecture

The agentic part is not merely a wrapper that gives the model web search.

Design a real research-control system with explicit state:

ResearchState = { question, assumptions, known_facts, unknowns, sources, evidence, hypotheses, experiments, results, contradictions, confidence, open_tasks, provenance, final_claims }

The controller should support actions such as:

- SEARCH
- READ
- EXTRACT
- COMPARE
- CALCULATE
- CODE
- RUN
- SIMULATE
- PROVE
- CHALLENGE
- HYPOTHESIZE
- VERIFY
- ABSTAIN
- REVISE
- WRITE

Research whether action selection should be learned, rule-based, or hybrid.

A strong candidate is a **hybrid deterministic shell + learned policy**: the shell enforces provenance and tool contracts; the learned components decide what to investigate next.

17. Research memory / evidence graph

Do not rely only on a vector database.

Investigate a structured evidence graph containing:

Paper -> Claim -> Evidence span -> Dataset -> Method -> Equation -> Experiment -> Result -> Limitation -> Follow-up hypothesis

Every generated research claim should ideally point to a provenance chain.

Study graph databases, relational evidence stores, vector retrieval, hybrid retrieval, temporal versioning, and citation graphs.

AIM should be able to answer not only:

"What does the system believe?"

but:

"Why does it believe this, what evidence supports it, what evidence contradicts it, and what would change its mind?"

18. Novel research direction to investigate

The most interesting candidate contribution is not merely "we used RLHF + RLVR + RLCD."

Investigate whether the core novelty could be:

Research-Loop Reinforcement Learning: *training a model whose policy is defined over research actions and evidence states, rather than over final text alone.*

Possible formulation:

state_t = research state action_t = research operation / hypothesis / tool call observation_t = evidence or tool result reward_t = verification + progress + calibration + human usefulness

Then investigate whether a research trajectory can be optimized as a sequential decision problem rather than as a single-response preference problem.

This could allow RLCD to operate at the **controller layer**, while RLVR evaluates scientific outcomes and RLHF evaluates the final human-facing artifact.

Treat this as a hypothesis to test, not as a predetermined novelty claim.

19. Required adversarial questions

Astra must answer these explicitly:

Does combining RLHF, RLVR, and RLCD actually improve research ability, or merely complicate training?

Can calibration be learned for claims whose truth cannot be observed immediately?

How should delayed scientific outcomes be handled?

Can a decision model meaningfully supervise a generative model?

When should the judge abstain?

How do we prevent the judge from becoming confidently wrong?

How do we stop the generator from learning to game the verifier?

How do we prevent reward hacking through verbose or superficially rigorous reasoning?

Does process reward improve genuine reasoning or merely style?

Does MoE specialization improve cross-domain research or fragment knowledge?

How much of research intelligence comes from pretraining versus post-training?

What training data most strongly determines scientific ability?

What is the real bottleneck: parameters, tokens, reward quality, verifier coverage, inference compute, or research-loop design?

Can CPU clusters meaningfully contribute to large-scale LLM development even if they cannot economically pretrain the final model?

What would count as evidence that AIM is a genuinely new research contribution rather than an assembly of known techniques?

20. Required deliverables from Astra

Save all outputs inside the existing AIM project folder. Do not put source documents inside the code/model directory unless explicitly needed.

Recommended structure:

AIM/ research/ literature/ architecture/ training/ rl/ data/ evaluation/ scaling/ hardware/ agentic-research/ open-questions/ documents/ research-charter/ architecture-spec/ training-spec/ evaluation-spec/ final-design/ model/ base/ checkpoints/ configs/ tokenizer/ code/ data/ training/ inference/ rl/ verifiers/ agent/ evaluation/ distributed/

Create, at minimum:

research/README.md

research/literature/literature-matrix.md

research/architecture/state-of-the-art-architecture-review.md

research/rl/rlhf-rlvr-rlcd-review.md

research/rl/reward-and-verifier-design.md

research/data/data-strategy.md

research/scaling/scaling-and-compute-analysis.md

research/hardware/vit-ml-lab-audit.md

```

research/agent-ic-research/research-loop-design.md
research/evaluation/evaluation-and-benchmark-plan.md
research/open-questions/open-research-questions.md
documents/final-design/AIM-Research-Intelligence-Model-Architecture-Spec.md
documents/final-design/AIM-Research-Intelligence-Model-Training-Spec.md
documents/final-design/AIM-Research-Intelligence-Model-Evaluation-Spec.md
documents/final-design/AIM-Research-Intelligence-Model-Research-Plan.md
documents/final-design/DECISIONS.md

```

The final design documents must contain:

- chosen architecture;
- rejected alternatives and why;
- parameter count(s);
- active parameter count for MoE if applicable;
- layer count;
- hidden dimension;
- attention heads / KV heads;
- FFN / expert configuration;
- context length strategy;
- tokenizer design;
- positional encoding;
- precision;
- optimizer;
- learning-rate schedule;
- batch / sequence strategy;
- data mixture;
- estimated tokens;
- pretraining objective;
- SFT objective;
- RLHF method;
- RLVR method(s);
- RLCD method;
- reward composition;
- verifier architecture;
- research-state representation;
- memory architecture;
- inference-time reasoning/search strategy;
- distributed-training topology;
- hardware assumptions;
- compute estimate;
- storage estimate;
- checkpoint strategy;
- evaluation matrix;
- failure modes;
- reproducibility plan.

21. Research protocol for Astra

Use this protocol throughout the investigation:

Source quality

Prefer:

Original paper / official technical report.

Official code/repository/model card.

Official engineering announcement.

Peer-reviewed secondary analysis.

High-quality independent technical analysis.

General web commentary only for discovery, not final technical claims.

Claim discipline

For every major technical claim, record:

- claim;
- source;
- date;
- evidence;
- confidence;
- whether independently reproduced;
- whether it is a paper result, engineering claim, or our own inference.

Date discipline

Research current material through **24 September 2026**.

Intellectual discipline

Do not converge early.

For major design choices, require:

- at least 3 credible alternatives;
- a comparison matrix;
- explicit trade-offs;
- uncertainty;
- what experiment would decide the question.

Engineering discipline

Do not implement a large component merely because it sounds good. First identify the smallest experiment that would falsify its necessity.

22. Current evidence anchors

Use these as starting points, then expand the literature:

- TypeSafe's September 15, 2026 announcement is the primary current source on Jev and RLCD.
- OpenAI's InstructGPT work establishes the classic RLHF pipeline with demonstrations, preference data, reward modeling, and PPO.
- DeepSeekMath introduced GRPO and demonstrated reinforcement learning for mathematical reasoning.
- DeepSeek-R1 demonstrated that RL can substantially elicit reasoning behavior and used a multi-stage post-training pipeline.

- DeepSeek-V3 demonstrates an extremely large sparse MoE architecture with 671B total and 37B active parameters.
- Qwen3 demonstrates a unified model family with thinking and non-thinking modes and both dense and MoE architectures.
- GLM-4.5 demonstrates a 355B-total / 32B-active MoE with multi-stage training and reasoning/agent/coding goals.
- Llama 4 Maverick demonstrates 400B total / 17B active MoE architecture.
- OLMo 3 demonstrates that fully open model development can include unusually detailed lifecycle disclosures.
- Process-reward research warns that naive absolute process rewards can introduce reward hacking or overvalue locally plausible but ultimately wrong reasoning.
- Muon and related optimizer work should be evaluated for training-efficiency implications rather than adopted automatically.

23. Final instruction

Do not return a shallow "recommended architecture" after reading a few articles.

Treat this as a genuine research program.

The desired outcome is a **defensible, experimentally testable design for a large research-intelligence model**, with a clear path from small reproducible experiments to a substantial model.

The final report should be written as though another ML research team will try to reproduce the work from your documents without talking to us.

Where evidence is weak, say so. Where a design is speculative, label it speculative. Where our ambition exceeds the available compute, quantify the gap. Where a smaller experiment can answer a major question, propose and prioritize that experiment.

Most importantly:

Do not optimize for sounding like a frontier AI lab. Optimize for discovering what actually works.

Appendix B — AIM Research Intelligence Model Charter (source supplied by project owner)

Included as project-source material. Requirements and claims in the source are not treated as research findings unless independently supported in the main report.

AIM

Research Intelligence Model

Research Charter, Architecture Hypotheses & Training Program

Version 0.1 | 24 September 2026

Executive Summary

AIM is a proposed large-model research program built around a simple thesis: reliable research intelligence requires multiple forms of learning that are poorly captured by any one training signal. The program will investigate a generative research model, a calibrated decision component, a verifier ensemble, and an explicit research-state loop. The core learning stack will experimentally combine human preference optimization (RLHF or strong alternatives), verifiable reinforcement learning (RLVR), and calibrated decision learning inspired by the September 2026 RLCD work from TypeSafe/Jev.

The project is deliberately not framed as “build a chatbot that knows many subjects.” The target capability is iterative research: define a question, retrieve evidence, generate competing hypotheses, derive predictions, run tools or experiments, verify claims, detect contradictions, update confidence, and write a research artifact whose claims retain provenance.

The scale ambition is intentionally large. The study will evaluate dense models from roughly 7B to 70B parameters and sparse MoE systems from roughly 100B to 400B+ total parameters, with final selection driven by actual hardware and compute profiling. Public frontier reports demonstrate that sparse architectures can expose very large total parameter counts while activating far fewer parameters per token; examples include DeepSeek-V3 (671B total / 37B active), GLM-4.5 (355B / 32B active), and Llama 4 Maverick (400B / 17B active). [2][4][5][6]

1. What We Are Actually Building

The product is best understood as a model family plus a research operating loop, rather than one monolithic neural network.

A large generative Researcher model that is strong at mathematics, science, code, scholarly writing, synthesis, hypothesis generation, and cross-domain reasoning.

A calibrated Decision/Judge component that makes structured decisions about evidence state, uncertainty, contradiction, continuation, retry, hypothesis selection, and verification status.

A verifier ensemble that can objectively test parts of the model's work through code execution, symbolic mathematics, theorem proving, simulations, statistics, and citation/source checking.

A research-state controller that turns the interaction into a sequential decision problem rather than a single prompt-response exchange.

A provenance-aware research memory/evidence graph that records claims, sources, evidence, derivations, experiments, results, and limitations.

2. The Central Research Hypothesis

The working hypothesis is that research ability improves when three otherwise different objectives are deliberately separated and then connected:

RLHF / preference optimization: human-facing usefulness, clarity, faithful instruction following, research writing and communication.

RLVR: objective correctness on tasks whose outcomes can be verified automatically.

RLCD: calibrated structured decisions rather than unconstrained prose confidence.

The project must test whether these signals work best sequentially, alternately, jointly, or in different model heads/components. The goal is not to combine fashionable acronyms into one loss; it is to determine what each signal is structurally good at and give it the right place in the architecture.

3. Why the RLHF + RLVR + RLCD Idea Is Interesting

RLHF is good at objectives that are hard to specify with deterministic rules, but its classic form depends on human preference labels and a reward model. InstructGPT is the canonical public example of demonstrations, preference comparisons, a learned reward model, and PPO-based policy optimization. [1]

RLVR is attractive because many research operations can be made objectively testable. DeepSeekMath introduced GRPO for mathematical reasoning, and DeepSeek-R1/R1-Zero showed that reinforcement learning can substantially elicit reasoning behaviors under verifiable rewards. [3][4] More recent work on process rewards shows why simply rewarding intermediate-looking reasoning can be dangerous: a process reward can favor locally plausible steps even when the final trajectory is wrong, creating reward-hacking pressure. [10]

RLCD is interesting for a different reason. TypeSafe's September 15, 2026 Jev announcement describes a model optimized for structured probabilistic decisions rather than free-form string generation, with RLCD presented as the training method. This suggests a potentially useful separation: generation can remain expressive while a decision subsystem handles calibrated choices, abstention, and control of the research loop. [7]

The important caveat is that constrained output is not the same as truth. A decision model can still select the wrong decision. "No hallucination" should therefore be interpreted narrowly as a property of the output contract, not as proof of universal factual correctness. [7]

4. Candidate Architecture

The current architecture hypothesis is a sparse MoE language model with an explicit research-control stack around it. Three variants should be studied before commitment.

A strong starting point is Variant C, but the research program must remain willing to collapse to A or B if experiments show that the additional machinery does not improve verified research performance.

5. Scale Program

The final model size must be chosen after an empirical hardware audit. A high parameter count without the compute to train it well is not automatically more impressive; quality, data volume, reward quality, and verification coverage matter as much as scale.

6. Compute Reality Check

A transparent first-order dense pretraining estimate is: $\text{training FLOPs} \approx 6 \times N_{\text{parameters}} \times N_{\text{training_tokens}}$. This is a useful order-of-magnitude model, not a universal exact formula. [9] For sparse MoE systems, per-token compute tracks active parameters more closely than total parameters, while memory and communication still track the full expert set.

A 32B dense model trained on 640B tokens is roughly $1.23\text{e}23$ training FLOPs.

A 70B dense model trained on 1.4T tokens is roughly $5.88\text{e}23$ FLOPs.

A 32B-active MoE trained on 10T tokens is roughly $1.92\text{e}24$ active-parameter FLOPs before accounting for routing/communication overhead.

At 1 PFLOP/s sustained, the last figure is on the order of decades, not days. This makes the current i9 cluster strategically valuable for data, simulation, verification, RL rollouts, evaluation and proxy training, but it also means full-scale pretraining must be benchmarked honestly and may require additional accelerator access. The project must quantify the actual gap instead of pretending "no time limit" makes compute free.

7. Training Program

Stage 0 - Hardware and distributed benchmark

Profile CPU, RAM, storage, network, any available GPUs, matrix multiplication, tokenization, inference, all-reduce, and MoE-style all-to-all.

Stage 1 - Data + tokenizer

Build licensed data ingestion, filtering, deduplication, contamination checks and domain-mixture tooling.

Stage 2 - Base pretraining

Run scaling-law experiments and train serious proxy models.

Stage 3 - Mid-training

Increase scientific, mathematical, code, long-context and research-process coverage.

Stage 4 - Research SFT

Teach research state, tool contracts, evidence records, hypothesis/result formats and scholarly communication.

Stage 5 - RLVR

Optimize code, math, symbolic, simulation and structured research environments against objective verifiers.

Stage 6 - RLHF / preference

Optimize usefulness, clarity, communication and researcher preferences.

Stage 7 - RLCD

Train calibrated decisions for evidence state, uncertainty, retry/continue, hypothesis choice and source selection.

Stage 8 - Integrated research loop

Combine model, controller, judge and verifiers as a sequential research environment.

Stage 9 - Adversarial evaluation

Continuously test factuality, calibration, reward hacking, shortcut learning and domain generalization.

8. Combined Objective - Research Program, Not One Magic Loss

A candidate conceptual reward is:

$$R_{\text{total}}(\tau) = w_h R_{\text{human}} + w_v R_{\text{verifiable}} + w_c R_{\text{calibrated}} + w_p R_{\text{process}} - w_k \text{KL}(\pi || \pi_{\text{ref}}) - \text{penalties}$$

This expression is deliberately provisional. The program must determine whether one scalar reward is even the right mathematical object. A vector-valued or constrained objective may be more appropriate because “human usefulness,” “objective correctness,” and “calibration” are not naturally commensurate quantities.

The mathematical appendix should cover autoregressive likelihood, preference modeling, PPO/GRPO, DAPO, process rewards, KL regularization, Brier score, log score, ECE and selective prediction/abstention.

9. Research-Agent Loop

The agentic layer should maintain explicit state rather than hiding the entire research process inside a prompt.

ResearchState = { question, assumptions, facts, unknowns, sources, evidence, hypotheses, experiments, results, contradictions, confidence, open_tasks, provenance, final_claims }

Candidate learned actions include SEARCH, READ, EXTRACT, COMPARE, CALCULATE, CODE, RUN, SIMULATE, PROVE, CHALLENGE, HYPOTHESIZE, VERIFY, ABSTAIN, REVISE and WRITE. A hybrid deterministic shell should enforce provenance and tool contracts while learned components decide what to investigate next.

10. Truth and Provenance Model

AIM should treat the epistemic state of a claim as first-class data. Suggested states are: VERIFIED, STRONGLY SUPPORTED, PARTIALLY SUPPORTED, UNCERTAIN, HYPOTHESIS, CONTRADICTED, UNVERIFIED and UNKNOWN/INSUFFICIENT EVIDENCE.

A claim should ideally carry a provenance chain: claim -> source -> evidence span -> derivation/transformation -> verification result -> confidence. The system should be able to explain not only what it believes, but why, what contradicts it, and what evidence could change its mind.

11. Data Strategy

Knowledge corpus: filtered web, high-quality educational content, open scientific literature, textbooks and documentation that are legally reusable.

Structured reasoning: mathematical derivations, proofs, algorithms, experiments, code and scientific worked examples.

Research process trajectories: question -> search -> evidence -> hypotheses -> tests -> conclusions.

Negative examples: confident errors, fabricated citations, invalid proofs, bad experiments and plausible-but-wrong reasoning.

Synthetic environments: procedurally generated mathematics, algorithms, science and research worlds with fully observable ground truth.

Data provenance and licensing must be tracked. “Publicly accessible” is not a sufficient criterion for training-data reuse.

12. Evaluation Philosophy

The evaluation program must avoid reducing the system to MMLU-like question answering. Research competence needs open-ended and trajectory-level tasks.

Factuality and source attribution.

Exact and derivational mathematics.

Code with hidden tests and runtime verification.

Scientific hypothesis generation and experiment design.

Literature synthesis and contradiction discovery.

Calibration using Brier score, log score, ECE and selective accuracy.

Long-horizon research tasks with evidence tracking.

Adversarial tests for fabricated citations, false premises and reward hacking.

Held-out problem generators and cross-domain generalization.

13. Research Risks

14. What Would Make AIM Genuinely Significant

The contribution should not be “we trained a big LLM.” Scale alone is not enough. Stronger candidates include a measurable improvement in research-loop performance, a new way of combining generative and calibrated decision policies, better verified reasoning under mixed rewards, a principled reward/verifier architecture, or a new training recipe that yields unusually strong cross-domain research ability for a given compute budget.

A convincing paper should include ablations that isolate the contribution of RLHF, RLVR, RLCD, verifier diversity, research-state memory, model size, and inference-time search. A system that only works because every component is turned on provides weaker causal evidence than a carefully controlled study.

15. Immediate Research Deliverables for Astra

Literature matrix covering foundation-model scaling, MoE, RLHF, RLVR, RLCD, calibration, reward modeling, verifiers, long-context methods and research agents.

Architecture review comparing at least three serious system designs.

Mathematical derivation and critique of the RLHF + RLVR + RLCD objective.

Compute/scaling report tied to measured VIT lab hardware.

Data strategy with source provenance and licensing notes.

Research-agent state machine and tool architecture.

Benchmark and evaluation specification.

Final architecture specification with chosen parameters and explicit rejected alternatives.

Training specification with staged objectives and RL schedule.

Decision log recording unresolved questions and what experiments will settle them.

16. Reference Anchors

17. Closing Principle

Do not optimize for sounding like a frontier AI lab. Optimize for discovering what actually works.

Mission: build a large, research-oriented language model system that can reason, investigate, verify, calibrate uncertainty, and produce defensible scientific work - not simply generate convincing text.

Variant	Core design	Strength	Question to test
A	Shared Transformer/MoE backbone + specialized decision head	Parameter sharing; compact judge integration	Does a head have enough representational separation?
B	Large Researcher model + separate calibrated Judge	Strong modularity and independent scaling	Does judge latency/quality become the bottleneck?
C	Large Researcher MoE + judge + verifier/router stack	Best separation of concerns; strong research-loop control	Can system complexity be justified by measurable gains?

Class	Illustrative scale	Purpose	Approx. BF16 weights	Training implication	Decision
Dense baseline	7B	Serious reproducibility baseline	~14 GB	Manageable proxy for full pipeline	Required
Dense mainline	14B	Substantial model	~28 GB	Strong research capability candidate	Evaluate
Dense large	32B	Major research model	~64 GB	Very large optimizer/activation footprint	Evaluate
Dense frontier-study	70B	Upper dense reference	~140 GB	Very high training cost	Theoretical/conditional
MoE mainline	100-200B total / 15-30B active	Large sparse system	~200-400 GB total	High communication + optimizer burden	Evaluate
MoE flagship	300-400B total / 25-40B active	Frontier-inspired research target	~600-800 GB total	Needs serious distributed hardware	Aspirational

Risk	Failure mode	Control
Reward hacking	Model learns verifier quirks rather than scientific correctness.	Multiple independent verifiers; hidden tests; adversarial verifier evaluation.
Judge failure	Calibration/decision model becomes a single point of failure.	Ensembles, abstention, uncertainty audits and periodic external evaluation.
Preference overoptimization	Model becomes polished but less truthful or diverse.	RLHF/DPO ablations, truthfulness checks, preference-collapse analysis.
Compute illusion	Large parameter count without sufficient training.	Scaling-law studies and hardware-grounded budgets.
Data contamination	Benchmarks or evaluation items appear in training.	Source lineage, deduplication, temporal and structural contamination checks.
Novelty inflation	System is a recombination of known techniques framed as a new algorithm.	Explicit contribution taxonomy and ablation-driven novelty claims.
Research theater	Excellent demos but weak scientific evidence.	Pre-registered experiments, negative results, reproducible configs and independent evaluation.

ID	Reference	Link
[1]	Ouyang et al., "Training language models to follow instructions with human feedback" / InstructGPT, OpenAI, 2022.	https://openai.com/index/instruction-following/
[2]	DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024. 671B total / 37B active; 14.8T pretraining tokens.	https://arxiv.org/abs/2412.19437
[3]	Shao et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," 2024.	https://arxiv.org/abs/2402.03300
[4]	Guo et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," 2025.	https://arxiv.org/abs/2501.12948
[5]	Yang et al., "Qwen3 Technical Report," 2025. Dense and MoE family up to 235B; thinking/non-thinking modes.	https://arxiv.org/abs/2505.09388
[6]	GLM-4.5 Team, "GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models," 2025. 355B total / 32B active.	https://arxiv.org/abs/2508.06471
[7]	TypeSafe AI, "Introducing System One Models & Jev," 15 September 2026. RLCD and structured probabilistic decisions.	https://typesafe.ai/blog/introducing-system-one-models-and-jev

ID	Reference	Link
[8]	Hoffmann et al., "Training Compute-Optimal Large Language Models," 2022.	https://arxiv.org/abs/2203.15556
[9]	Kaplan et al., "Scaling Laws for Neural Language Models," 2020.	https://arxiv.org/abs/2001.08361
[10]	Rezaei et al., "LLM Reasoning with Process Rewards for Outcome-Guided Steps," 2026. Process-reward failure modes and outcome-conditioned process rewards.	https://arxiv.org/abs/2604.02341
[11]	Yu et al., "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," 2025.	https://arxiv.org/abs/2503.14476
[12]	Rafailov et al., "Direct Preference Optimization," 2023.	https://arxiv.org/abs/2305.18290
[13]	Yuan et al., "Self-Rewarding Language Models," 2024.	https://arxiv.org/abs/2401.10020
[14]	Bai et al., "Constitutional AI: Harmlessness from AI Feedback," 2022.	https://arxiv.org/abs/2212.08073
[15]	Liu et al., "Muon is Scalable for LLM Training," 2025.	https://arxiv.org/abs/2502.16982
[16]	Team OLMo, "Olmo 3," 2025. Fully open 7B/32B model lifecycle.	https://arxiv.org/abs/2512.13961